

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	10
2 МЕТОД РЕШЕНИЯ.....	12
3 ОПИСАНИЕ АЛГОРИТМОВ.....	17
3.1 Алгоритм метода set_connection класса cl_base.....	17
3.2 Алгоритм метода delete_connection класса cl_base.....	18
3.3 Алгоритм метода emit_signal класса cl_base.....	19
3.4 Алгоритм метода get_abs_path класса cl_base.....	20
3.5 Алгоритм метода get_class_index класса cl_base.....	21
3.6 Алгоритм метода set_state_for_branch класса cl_base.....	21
3.7 Алгоритм метода signal_f класса cl_application.....	22
3.8 Алгоритм метода handler_f класса cl_application.....	22
3.9 Алгоритм метода get_class_index класса cl_application.....	23
3.10 Алгоритм метода exec_app класса cl_application.....	23
3.11 Алгоритм метода signal_f класса cl_2.....	27
3.12 Алгоритм метода handler_f класса cl_2.....	27
3.13 Алгоритм метода get_class_index класса cl_2.....	27
3.14 Алгоритм метода get_class_index класса cl_3.....	28
3.15 Алгоритм метода get_class_index класса cl_4.....	28
3.16 Алгоритм метода get_class_index класса cl_5.....	29
3.17 Алгоритм метода get_class_index класса cl_6.....	29
3.18 Алгоритм метода handler_f класса cl_3.....	29
3.19 Алгоритм метода handler_f класса cl_4.....	30
3.20 Алгоритм метода handler_f класса cl_5.....	30
3.21 Алгоритм метода handler_f класса cl_6.....	31

3.22 Алгоритм метода signal_f класса cl_3.....	31
3.23 Алгоритм метода signal_f класса cl_4.....	32
3.24 Алгоритм метода signal_f класса cl_5.....	32
3.25 Алгоритм метода signal_f класса cl_6.....	33
3.26 Алгоритм функции main.....	33
3.27 Алгоритм функции num_to_signal.....	34
3.28 Алгоритм функции num_to_handler.....	34
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	36
5 КОД ПРОГРАММЫ.....	58
5.1 Файл cl_2.cpp.....	58
5.2 Файл cl_2.h.....	59
5.3 Файл cl_3.cpp.....	59
5.4 Файл cl_3.h.....	60
5.5 Файл cl_4.cpp.....	60
5.6 Файл cl_4.h.....	61
5.7 Файл cl_5.cpp.....	61
5.8 Файл cl_5.h.....	62
5.9 Файл cl_6.cpp.....	62
5.10 Файл cl_6.h.....	62
5.11 Файл cl_application.cpp.....	63
5.12 Файл cl_application.h.....	70
5.13 Файл cl_base.cpp.....	70
5.14 Файл cl_base.h.....	75
5.15 Файл main.cpp.....	77
6 ТЕСТИРОВАНИЕ.....	78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	80

1 ПОСТАНОВКА ЗАДАЧИ

Реализовать механизм взаимодействия объектов с использованием сигналов и обработчиков, с передачей вместе сигналом текстового сообщения (строковой переменной).

Для организации взаимосвязи по механизму сигналов и обработчиков в базовый класс добавить три метода:

- установления связи между сигналом текущего объекта и обработчиком целевого объекта;
- удаления (разрыва) связи между сигналом текущего объекта и обработчиком целевого объекта;
- выдачи сигнала от текущего объекта с передачей строковой переменной.

Включенный объект может выдать или обработать сигнал.

Методу установки связи передать указатель на метод сигнала текущего объекта, указатель на целевой объект и указатель на метод обработчика целевого объекта.

Методу удаления (разрыва) связи передать указатель на метод сигнала текущего объекта, указатель на целевой объект и указатель на метод обработчика целевого объекта.

Методу выдачи сигнала передать указатель на метод сигнала и строковую переменную. В данном методе реализовать алгоритм:

1. Если текущий объект отключен, то выход, иначе к пункту 2.
2. Вызов метода сигнала с передачей строковой переменной по ссылке.
3. Цикл по всем связям сигнал-обработчик текущего объекта:
 - 3.1. Если в очередной связи сигнал-обработчик участвует метод сигнала, переданный по параметру, то проверить готовность целевого объекта. Если целевой объект готов, то вызвать метод обработчика

целевого объекта указанной в связи и передать в качестве аргумента строковую переменную по значению.

4. Конец цикла.

Для приведения указателя на метод сигнала и на метод обработчика использовать параметризованное макроопределение препроцессора.

В базовый класс добавить метод определения абсолютной пути до текущего объекта. Этот метод возвращает абсолютный путь текущего объекта.

Состав и иерархия объектов строится посредством ввода исходных данных. Ввод организован как в версии № 3 курсовой работы. Если при построении дерева иерархии возникает ситуация дуближа имен среди починенных у текущего головного объекта, то новый объект не создается.

Система содержит объекты шести классов с номерами: 1, 2, 3, 4, 5, 6. Классу корневого объекта соответствует номер 1. В каждом производном классе реализовать один метод сигнала и один метод обработчика.

Каждый метод сигнала с новой строки выводит:

Signal from «абсолютная координата объекта»

Каждый метод сигнала добавляет переданной по параметру строке текста номер класса принадлежности текущего объекта по форме:

«пробел»(class: «номер класса»)

Каждый метод обработчика с новой строки выводит:

Signal to «абсолютная координата объекта» Text: «переданная строка»

Моделировать работу системы, которая выполняет следующие команды с параметрами:

- EMIT «координата объекта» «текст» – выдает сигнал от заданного по координате объекта;
- SET_CONNECT «координата объекта выдающего сигнал» «координата

целевого объекта» – устанавливает связь;

- DELETE_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – удаляет связь;
- SET_CONDITION «координата объекта» «значение состояния» – устанавливает состояние объекта.
- END – завершает функционирование системы (выполнение программы).

Реализовать алгоритм работы системы:

- в методе построения системы:
 - о построение дерева иерархии объектов согласно вводу;
 - о ввод и построение множества связей сигнал-обработчик для заданных пар объектов.
- в методе отработки системы:
 - о привести все объекты в состоянии готовности;
 - о цикл до признака завершения ввода:
 - ввод наименования объекта и текста сообщения;
 - вызов сигнала заданного объекта и передача в качестве аргумента строковой переменной, содержащей текст сообщения.
 - о конец цикла.

Допускаем, что все входные данные вводятся синтаксически корректно. Контроль корректности входных данных можно реализовать для самоконтроля работы программы. Не оговоренные, но необходимые функции и элементы классов добавляются разработчиком.

1.1 Описание входных данных

В методе построения системы.

Множество объектов, их характеристики и расположение на дереве

иерархии. Структура данных для ввода согласно изложенному в версии № 3 курсовой работы.

После ввода состава дерева иерархии построчно вводится:

«координата объекта выдающего сигнал» «координата целевого объекта»

Ввод информации для построения связей завершается строкой, которая содержит:

«end_of_connections»

В методе запуска (отработки) системы построчно вводятся множество команд в производном порядке:

- EMIT «координата объекта» «текст» – выдать сигнал от заданного по координате объекта;
- SET_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – установка связи;
- DELETE_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – удаление связи;
- SET_CONDITION «координата объекта» «значение состояния» – установка состояния объекта.
- END – завершить функционирование системы (выполнение программы).

Команда END присутствует обязательно.

Если координата объекта задана некорректно, то соответствующая операция не выполняется и с новой строки выдается сообщение об ошибке.

Если не найден объект по координате:

Object «координата объекта» not found

Если не найден целевой объект по координате:

Handler object «координата целевого объекта» not found

Пример ввода:

```
appls_root
/ object_s1 3
/ object_s2 2
/object_s2 object_s4 4
/ object_s13 5
/object_s2 object_s6 6
/object_s1 object_s7 2
endtree
/object_s2/object_s4 /object_s2/object_s6
/object_s2 /object_s1/object_s7
/ /object_s2/object_s4
/object_s2/object_s4 /
end_of_connections
EMIT /object_s2/object_s4 Send message 1
EMIT /object_s2/object_s4 Send message 2
EMIT /object_s2/object_s4 Send message 3
EMIT /object_s1 Send message 4
END
```

1.2 Описание выходных данных

Первая строка:

Object tree

Со второй строки вывести иерархию построенного дерева.

Далее, построчно, если отработал метод сигнала:

Signal from «абсолютная координата объекта»

Если отработал метод обработчика:

Signal to «абсолютная координата объекта» Text: «переданная строка»

Пример вывода:

```
Object tree
appls_root
  object_s1
    object_s7
  object_s2
    object_s4
    object_s6
  object_s13
Signal from /object_s2/object_s4
Signal to /object_s2/object_s6 Text: Send message 1 (class: 4)
Signal to / Text: Send message 1 (class: 4)
Signal from /object_s2/object_s4
```

Signal to /object_s2/object_s6 Text: Send message 2 (class: 4)
Signal to / Text: Send message 2 (class: 4)
Signal from /object_s2/object_s4
Signal to /object_s2/object_s6 Text: Send message 3 (class: 4)
Signal to / Text: Send message 3 (class: 4)
Signal from /object_s1

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `target` класса `cl_base*` предназначен для хранения указателя на объект обработки;
- объект `actual_object` класса `cl_base*` предназначен для хранения указателя на объект над которым происходит взаимодействие;
- объект `destination_object` класса `cl_base*` предназначен для хранения указателя на объект обработки;
- объект `parent` класса `cl_base*` предназначен для хранения указателя на объект над которым происходит взаимодействие;
- функция `num_to_signal` для переводит числовое значение в объект типа `SIGNAL_D`;
- функция `num_to_handler` для переводит числовое значение в объект типа `HANDLER_D`;
- библиотека `iostream`;
- библиотека `vector`;
- библиотека `stack`;
- библиотека `queue`;
- объект стандартного потока вывода `cout`;
- объект стандартного потока ввода `cin`;
- операторы динамического выделения/освобождения памяти `new/delete`;
- оператор цикла со счетчиком `for`;
- оператор цикла с предусловием `while`;
- условные операторы ветвления `if/else`;
- оператор получения /разыменовывания указателя `*`;
- оператор получения ссылки/адреса `&`;

- оператор инкремента;
- арифметический оператор сложения;
- оператор .;
- оператор ->;
- оператор ->*.

Класс `cl_base`:

- свойства/поля:
 - поле динамический массив содержащий связи:
 - наименование — `connections`;
 - тип — `vector <o_sh*>`;
 - модификатор доступа — `private`;
- функционал:
 - метод `set_connection` — установка связи между объектами;
 - метод `delete_connection` — удаление связи между объектами;
 - метод `emit_signal` — задание значения методу;
 - метод `get_abs_path` — получение абсолютного пути объекта;
 - метод `get_class_index` — получение номера класса;
 - метод `set_state_for_branch` — установка значения состояния для дерева.

Класс `cl_application`:

- функционал:
 - метод `signal_f` — выводит откуда и куда пришел сигнал;
 - метод `handler_f` — выводит сообщение куда был отправлен объект и отправленное сообщение;
 - метод `get_class_index` — получение номера класса;
 - метод `exes_app` — обработка и вывод иерархии.

Класс `cl_2`:

- функционал:
 - метод `signal_f` — выводит откуда и куда пришел сигнал;
 - метод `handler_f` — выводит сообщение куда был отправлен объект и отправленное сообщение;
 - метод `get_class_index` — получение номера класса.

Класс `cl_3`:

- функционал:
 - метод `signal_f` — выводит откуда и куда пришел сигнал;
 - метод `handler_f` — выводит сообщение куда был отправлен объект и отправленное сообщение;
 - метод `get_class_index` — получение номера класса.

Класс `cl_4`:

- функционал:
 - метод `signal_f` — выводит откуда и куда пришел сигнал;
 - метод `handler_f` — выводит сообщение куда был отправлен объект и отправленное сообщение;
 - метод `get_class_index` — получение номера класса.

Класс `cl_5`:

- функционал:
 - метод `signal_f` — выводит откуда и куда пришел сигнал;
 - метод `handler_f` — выводит сообщение куда был отправлен объект и отправленное сообщение;
 - метод `get_class_index` — получение номера класса.

Класс `cl_6`:

- функционал:
 - метод `signal_f` — выводит откуда и куда пришел сигнал;
 - метод `handler_f` — выводит сообщение куда был отправлен объект и

отправленное сообщение;

о метод `get_class_index` — получение номера класса.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_base			основной класс в иерархии	
		cl_application	public		2
		cl_2	public		3
		cl_3	public		4
		cl_4	public		5
		cl_5	public		6
		cl_6	public		7
2	cl_application			производный класс реализующий функционал моделирования, обработки и вывода иерархии объектов	
3	cl_2			один из производных классов реализующих работу метода обработчика и отправки сигнала	
4	cl_3			один из производных классов реализующих работу метода обработчика и отправки сигнала	
5	cl_4			один из производных классов реализующих работу метода обработчика и отправки сигнала	
6	cl_5			один из производных классов реализующих работу метода обработчика и отправки сигнала	
7	cl_6			один из производных классов реализующих работу метода	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
				обработчика и отправки сигнала	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода `set_connection` класса `cl_base`

Функционал: установка связи между объектами.

Параметры: `TYPE_SIGNAL`, `p_signal`, указатель на метод отправки сигнала, `cl_base*`, `p_target`, указатель на объект класса, `TYPE_HANDLER`, `p_handler`, указатель на метод обработки сигнала.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода `set_connection` класса `cl_base`

№	Предикат	Действия	№ перехода
1		объявление указателя <code>p_value</code> на структуру <code>o_sh</code>	2
2		инициализация переменной <code>i</code> со значением 0	3
3	<code>i</code> меньше возвращаемого методом <code>size</code> для <code>connections</code> значения		4
			∅
4	элемент <code>p_signal</code> <code>i</code> элемента <code>connections</code> равен параметру <code>p_signal</code> и элемент <code>p_target</code> <code>i</code> элемента <code>connections</code> равен параметру <code>p_target</code> и элемент <code>p_handler</code> <code>i</code> элемента <code>connections</code> равен параметру	выход из метода	∅

№	Предикат	Действия	№ перехода
	p_handler		
		выделение памяти под структуру o_sh для переменной p_value	5
5		присвоение элементу p_signal структуры p_value параметра p_signal	6
6		присвоение элементу p_target структуры p_value параметра p_target	7
7		присвоение элементу p_handler структуры p_value параметра p_handler	8
8		вызов метода push_back для connections с параметром p_value	9
9		вызов оператора инкремента для i	4

3.2 Алгоритм метода delete_connection класса cl_base

Функционал: удаление связи между объектами.

Параметры: TYPE_SIGNAL, p_signal, указатель на метод отправки сигнала, cl_base*, p_target, указатель на объект класса, TYPE_HANDLER, p_handler, указатель на метод обработки сигнала.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода delete_connection класса cl_base

№	Предикат	Действия	№ перехода
1	итератор it не равен connections.end()		2
			∅
2	элемент p_signal указателя it равен параметру p_signal и	вызов оператор delete для указателя it	3

№	Предикат	Действия	№ перехода
	элемент p_target указателя it равен параметру p_target и элемент p_handler указателя it равен параметру p_handler		
		вызов оператора инкремента для it	1
3		присвоение it значения возвращаемого методом erase с параметром it для connections	4
4		выход из метода	∅

3.3 Алгоритм метода emit_signal класса cl_base

Функционал: задание значения методу.

Параметры: TYPE_SIGNAL, p_signal, указатель на метод отправки сигнала, string&, message, передаваемое сообщение.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода emit_signal класса cl_base

№	Предикат	Действия	№ перехода
1	значение state равно 0	выход из метода	∅
		вызов метода p_signal с параметром message	2
2	прохождение коллекции connections с помощью структуры connection		3
			∅
3	элемент p_signal структуры connection равен параметру p_signal и элемент p_handler структуры connection равен	для элемента p_target структуры connection применили оператор ->* с методом p_handler с параметром message структуры connection	2

№	Предикат	Действия	№ перехода
	параметру p_handler		
			2

3.4 Алгоритм метода get_abs_path класса cl_base

Функционал: получение абсолютного пути объекта.

Параметры: нет.

Возвращаемое значение: string, абсолютный путь к объекту.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода get_abs_path класса cl_base

№	Предикат	Действия	№ перехода
1		объявление строковой переменной path	2
2		объявление адаптера контейнера stack ids	3
3		инициализация объекта root со значением this	4
4	значение возвращаемое методом get_pointer() вызванным для объекта root не равно nullptr	вызов метода push с параметром значения возвращаемого методом get_name() вызванным для root для контейнера ids	5
			6
5		вызов метода get_pointer() для объекта root	4
6	возвращаемое значение empty() вызванного для ids равно false	добавление к path '/' и значения возвращаемого методом top() вызванного для ids	7
			8
7		вызов метода pop() для ids	6
8	возвращаемое значение empty() вызванного для path равно true	возврат ""	∅

№	Предикат	Действия	№ перехода
		возврат path	∅

3.5 Алгоритм метода `get_class_index` класса `cl_base`

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода `get_class_index` класса `cl_base`

№	Предикат	Действия	№ перехода
1		возврат 0	∅

3.6 Алгоритм метода `set_state_for_branch` класса `cl_base`

Функционал: установка значения состояния для дерева.

Параметры: целый, `new_state`, новое состояние для объекта.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода `set_state_for_branch` класса `cl_base`

№	Предикат	Действия	№ перехода
1	возвращаемое значение <code>get_pointer()</code> не равно <code>nullptr</code> и элемент <code>state</code> возвращаемого значения <code>get_pointer()</code> равен нулю	выход из метода	∅
		вызов метода <code>set_state</code> с параметром <code>new_state</code>	2
2		инициализация целочисленной переменной <code>i</code> со	3

№	Предикат	Действия	№ перехода
		значением 0	
3	i меньше возвращаемого методом size() вызванного для subordinate_objects значения	вызов метода set_state_for_branch с параметром new_state для i элемента subordinate_objects	4
			∅
4		вызов оператора инкремента для переменной i	3

3.7 Алгоритм метода signal_f класса cl_application

Функционал: выводит откуда и куда пришел сигнал.

Параметры: string&,msg,сообщение для передачи.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода signal_f класса cl_application

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal from ", вызов метода get_abs_path()	2
2		добавление значению перменной msg " (class: ", вызов метода to string с параметром get_class_index() ,")"	∅

3.8 Алгоритм метода handler_f класса cl_application

Функционал: выводит сообщение куда был отправлен объект и отправленное сообщение.

Параметры: строковый, msg, сообщение для обработки.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *handler_f* класса *cl_application*

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal to ", вызов метода <i>get_abs_path()</i> , " Text: ", <i>msg</i>	Ø

3.9 Алгоритм метода *get_class_index* класса *cl_application*

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода *get_class_index* класса *cl_application*

№	Предикат	Действия	№ перехода
1		возврат 1	Ø

3.10 Алгоритм метода *exes_app* класса *cl_application*

Функционал: обработка и вывод иерархии.

Параметры: нет.

Возвращаемое значение: целый, индикатор корректности выполнения программы.

Алгоритм метода представлен в таблице 11.

Таблица 11 – Алгоритм метода *exes_app* класса *cl_application*

№	Предикат	Действия	№ перехода
1		объявление строковых переменных <i>action</i> , <i>coordinate</i> , <i>information</i>	2
2		объявление указателя на объект <i>destination_object</i> класса <i>cl_base*</i>	3

№	Предикат	Действия	№ перехода
3		объявление указателя на объект actual_object класса cl_base*	4
4		объявление целочисленной переменной state	5
5		вызов метода set_state_for_branch для объекта this с параметром 1	6
6		объявление переменной p_signal пользовательского типа TYPE_SIGNAL	7
7		объявление переменной p_handler пользовательского типа TYPE_HANDLER	8
8		вызов метода rout для объекта this	9
9		ввод значения для переменной action	10
10	action не равна "END"	ввод значения для переменной coordinate	11
			∅
11		присвоение переменной actual_object указателя возвращаемого методом find_object_by_coord с параметром coordinate вызванного для объекта this	12
12	actual_object равен nullptr	вывод перехода на новую строку, "Object ", coordinate, " not found"	13
			15
13		ввод значения для переменной action	14
14		вызов оператора continue	10
15	action равна "EMIT"	присвоение переменной information введенной строки	16
			18
16		инициализация переменной num_class со значением возвращаемым методом get_class_index вызванным для объекта actual_object	17
17		вызов метода emit_signal с параметрами num_to_signal, information для объекта	18

№	Предикат	Действия	№ перехода
		actual_object	
18	action равна "SET_CONNECT"	ввод значения для переменной coordinate	19
			26
19		присвоение переменной destination_object указателя возвращаемого методом find_object_by_coord с параметром coordinate вызванного для объекта this	20
20	destination_object равен nullptr	вывод перехода на новую строку, "Handler object", coordinate, " not found"	21
			23
21		ввод значения для переменной action	22
22		вызов оператора continue	10
23		присвоение переменной p_signal значения возвращаемого методом num_to_signal с параметром значения возвращаемого вызванным для объекта actual_object метода get_class_index вызванного для объекта this	24
24		присвоение переменной p_handler значения возвращаемого методом num_to_handler с параметром значения возвращаемого вызванным для объекта destination_object метода get_class_index index вызванного для объекта this	25
25		вызов метода set_connection с параметрами p_signal, destination_object, p_handler для объекта actual_object	26
26	action равна "DELETE_CONNECT"	ввод значения для переменной coordinate	27
			34

№	Предикат	Действия	№ перехода
27		присвоение переменной destination_object указателя возвращаемого методом find_object_by_coord с параметром coordinate вызванного для объекта this	28
28	destination_object равен nullptr	вывод перехода на новую строку, "Handler object", coordinate, " not found"	29
			31
29		ввод значения для переменной action	30
30		вызов оператора continue	10
31		присвоение переменной p_signal значения возвращаемого методом num_to_signal с параметром значения возвращаемого вызванным для объекта actual_object метода get_class_index вызванного для объекта this	32
32		присвоение переменной p_handler значения возвращаемого методом num_to_handler с параметром значения возвращаемого вызванным для объекта destination_object метода get_class_index index вызванного для объекта this	33
33		вызов метода delete_connections с параметрами p_signal, destination_object, p_handler для объекта actual_object	34
34	action равна "SET_CONDITION"	ввод значения для переменной state	35
			36
35		вызов метода set_state с параметром state для объекта actual_object	36
36		ввод значения для переменной action	10

3.11 Алгоритм метода `signal_f` класса `cl_2`

Функционал: выводит откуда и куда пришел сигнал.

Параметры: `string&,msg`,сообщение для передачи.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода `signal_f` класса `cl_2`

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal from ", вызов метода <code>get_abs_path()</code>	2
2		добавление значению переменной <code>msg</code> " (class: ", вызов метода <code>to string</code> с параметром <code>get_class_index()</code> ,")"	Ø

3.12 Алгоритм метода `handler_f` класса `cl_2`

Функционал: выводит сообщение куда был отправлен объект и отправленное сообщение.

Параметры: нет.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода `handler_f` класса `cl_2`

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal to ", вызов метода <code>get_abs_path()</code> , " Text: ", <code>msg</code>	Ø

3.13 Алгоритм метода `get_class_index` класса `cl_2`

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 14.

Таблица 14 – Алгоритм метода *get_class_index* класса *cl_2*

№	Предикат	Действия	№ перехода
1		возврат 2	Ø

3.14 Алгоритм метода *get_class_index* класса *cl_3*

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 15.

Таблица 15 – Алгоритм метода *get_class_index* класса *cl_3*

№	Предикат	Действия	№ перехода
1		возврат 3	Ø

3.15 Алгоритм метода *get_class_index* класса *cl_4*

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 16.

Таблица 16 – Алгоритм метода *get_class_index* класса *cl_4*

№	Предикат	Действия	№ перехода
1		возврат 4	Ø

3.16 Алгоритм метода `get_class_index` класса `cl_5`

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 17.

Таблица 17 – Алгоритм метода `get_class_index` класса `cl_5`

№	Предикат	Действия	№ перехода
1		возврат 5	Ø

3.17 Алгоритм метода `get_class_index` класса `cl_6`

Функционал: получение номера класса.

Параметры: нет.

Возвращаемое значение: целый, номер класса.

Алгоритм метода представлен в таблице 18.

Таблица 18 – Алгоритм метода `get_class_index` класса `cl_6`

№	Предикат	Действия	№ перехода
1		возврат 6	Ø

3.18 Алгоритм метода `handler_f` класса `cl_3`

Функционал: выводит сообщение куда был отправлен объект и отправленное сообщение.

Параметры: нет.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 19.

Таблица 19 – Алгоритм метода *handler_f* класса *cl_3*

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal to ", вызов метода <i>get_abs_path()</i> , " Text: ", <i>msg</i>	Ø

3.19 Алгоритм метода *handler_f* класса *cl_4*

Функционал: выводит сообщение куда был отправлен объект и отправленное сообщение.

Параметры: нет.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 20.

Таблица 20 – Алгоритм метода *handler_f* класса *cl_4*

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal to ", вызов метода <i>get_abs_path()</i> , " Text: ", <i>msg</i>	Ø

3.20 Алгоритм метода *handler_f* класса *cl_5*

Функционал: выводит сообщение куда был отправлен объект и отправленное сообщение.

Параметры: нет.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 21.

Таблица 21 – Алгоритм метода *handler_f* класса *cl_5*

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal to ", вызов метода	Ø

№	Предикат	Действия	№ перехода
		get_abs_path(), " Text: ", msg	

3.21 Алгоритм метода handler_f класса cl_6

Функционал: выводит сообщение куда был отправлен объект и отправленное сообщение.

Параметры: нет.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 22.

Таблица 22 – Алгоритм метода handler_f класса cl_6

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal to ", вызов метода get_abs_path(), " Text: ", msg	Ø

3.22 Алгоритм метода signal_f класса cl_3

Функционал: выводит откуда и куда пришел сигнал.

Параметры: string&,msg,сообщение для передачи.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 23.

Таблица 23 – Алгоритм метода signal_f класса cl_3

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal from ", вызов метода get_abs_path()	2
2		добавление значению перменной msg " (class: ", вызов метода to string с параметром get_class_index() ,")"	Ø

3.23 Алгоритм метода `signal_f` класса `cl_4`

Функционал: выводит откуда и куда пришел сигнал.

Параметры: `string&,msg`,сообщение для передачи.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 24.

Таблица 24 – Алгоритм метода `signal_f` класса `cl_4`

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal from ", вызов метода <code>get_abs_path()</code>	2
2		добавление значению переменной <code>msg</code> " (class: ", вызов метода <code>to string</code> с параметром <code>get_class_index()</code> ,")"	Ø

3.24 Алгоритм метода `signal_f` класса `cl_5`

Функционал: выводит откуда и куда пришел сигнал.

Параметры: `string&,msg`,сообщение для передачи.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 25.

Таблица 25 – Алгоритм метода `signal_f` класса `cl_5`

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal from ", вызов метода <code>get_abs_path()</code>	2
2		добавление значению переменной <code>msg</code> " (class: ", вызов метода <code>to string</code> с параметром <code>get_class_index()</code> ,")"	Ø

3.25 Алгоритм метода `signal_f` класса `cl_6`

Функционал: выводит откуда и куда пришел сигнал.

Параметры: `string&,msg`,сообщение для передачи.

Возвращаемое значение: ничего не возвращает.

Алгоритм метода представлен в таблице 26.

Таблица 26 – Алгоритм метода `signal_f` класса `cl_6`

№	Предикат	Действия	№ перехода
1		вывод перехода на новую строку, "Signal from ", вызов метода <code>get_abs_path()</code>	2
2		добавление значению переменной <code>msg</code> " (class: ", вызов метода <code>to string</code> с параметром <code>get_class_index()</code> ,")"	Ø

3.26 Алгоритм функции `main`

Функционал: главная функция программы.

Параметры: нет.

Возвращаемое значение: целый, индикатор корректности выполнения работы.

Алгоритм функции представлен в таблице 27.

Таблица 27 – Алгоритм функции `main`

№	Предикат	Действия	№ перехода
1		создание объекта <code>ob_cl_application</code> класса <code>cl_application</code> с помощью параметризованного конструктора с параметром <code>nullptr</code>	2
2		вызов метода <code>build_tree_objects</code> для объекта <code>ob_cl_application</code>	3
3		вызов метода <code>exec_app</code> для объекта <code>ob_cl_application</code>	Ø

3.27 Алгоритм функции num_to_signal

Функционал: преобразует целочисленное значение в пользовательский тип в зависимости от номера класса.

Параметры: целый, num, номер класса.

Возвращаемое значение: TYPE_SIGNAL, преобразованное значение.

Алгоритм функции представлен в таблице 28.

Таблица 28 – Алгоритм функции num_to_signal

№	Предикат	Действия	№ перехода
1	num равна 1	преобразование в тип SIGNAL_D метода signal_f класса cl_application	Ø
	num равна 2	преобразование в тип SIGNAL_D метода signal_f класса cl_2	Ø
	num равна 3	преобразование в тип SIGNAL_D метода signal_f класса cl_2	Ø
	num равна 4	преобразование в тип SIGNAL_D метода signal_f класса cl_2	Ø
	num равна 5	преобразование в тип SIGNAL_D метода signal_f класса cl_2	Ø
	num равна 6	преобразование в тип SIGNAL_D метода signal_f класса cl_2	Ø
		возврат nullptr	Ø

3.28 Алгоритм функции num_to_handler

Функционал: преобразует целочисленное значение в пользовательский тип в зависимости от номера класса.

Параметры: целый, num, номер класса.

Возвращаемое значение: TYPE_HANDLER, преобразованное значение.

Алгоритм функции представлен в таблице 29.

Таблица 29 – Алгоритм функции num_to_handler

№	Предикат	Действия	№ перехода
1	num равна 1	преобразование в тип HANDLER_D метода handler_f класса cl_application	Ø
	num равна 2	преобразование в тип HANDLER_D метода handler_f класса cl_2	Ø
	num равна 3	преобразование в тип HANDLER_D метода handler_f класса cl_3	Ø
	num равна 4	преобразование в тип HANDLER_D метода handler_f класса cl_4	Ø
	num равна 5	преобразование в тип HANDLER_D метода handler_f класса cl_5	Ø
	num равна 6	преобразование в тип HANDLER_D метода handler_f класса cl_6	Ø
		возврат nullptr	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-22.

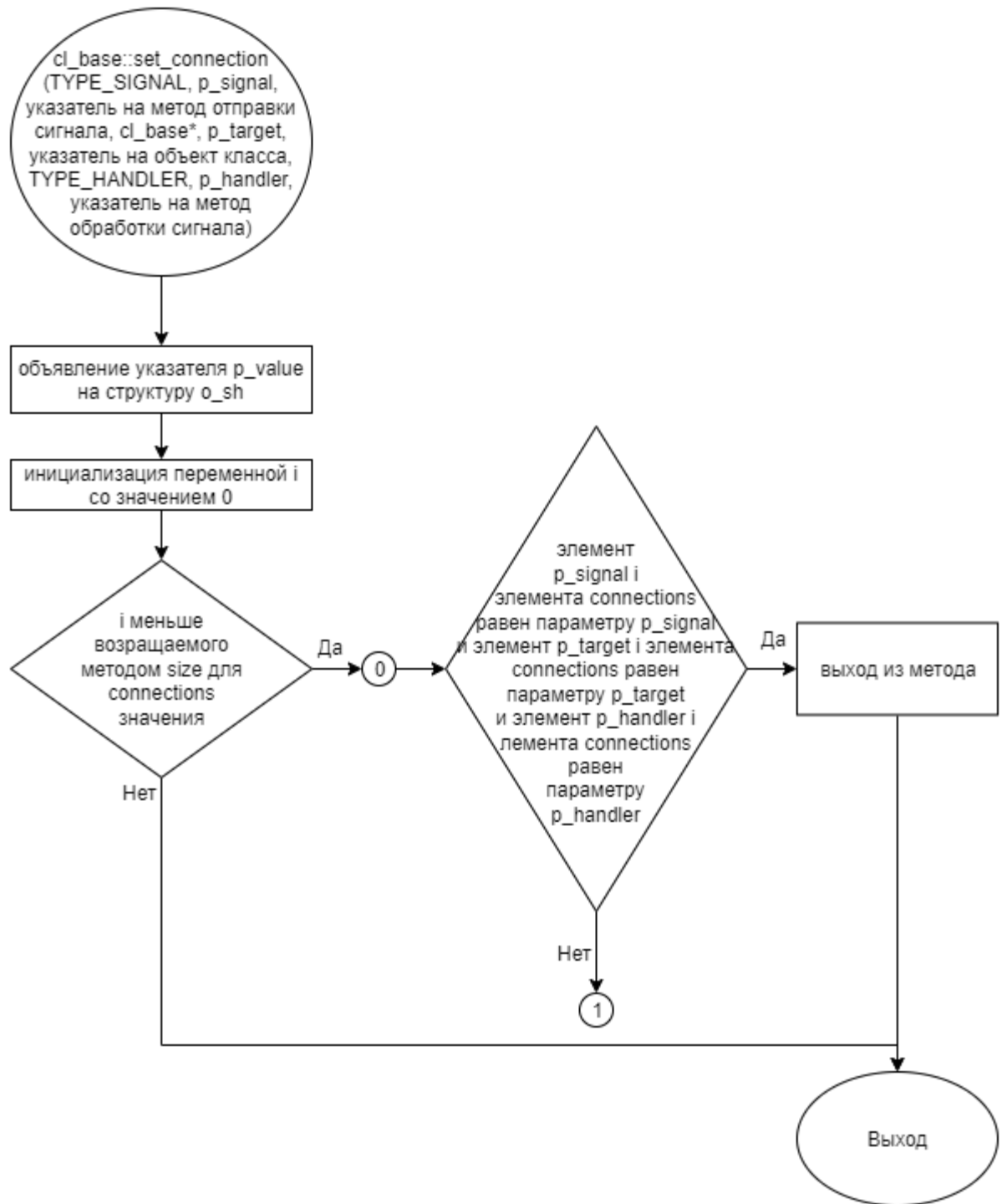


Рисунок 1 – Блок-схема алгоритма



Рисунок 2 – Блок-схема алгоритма

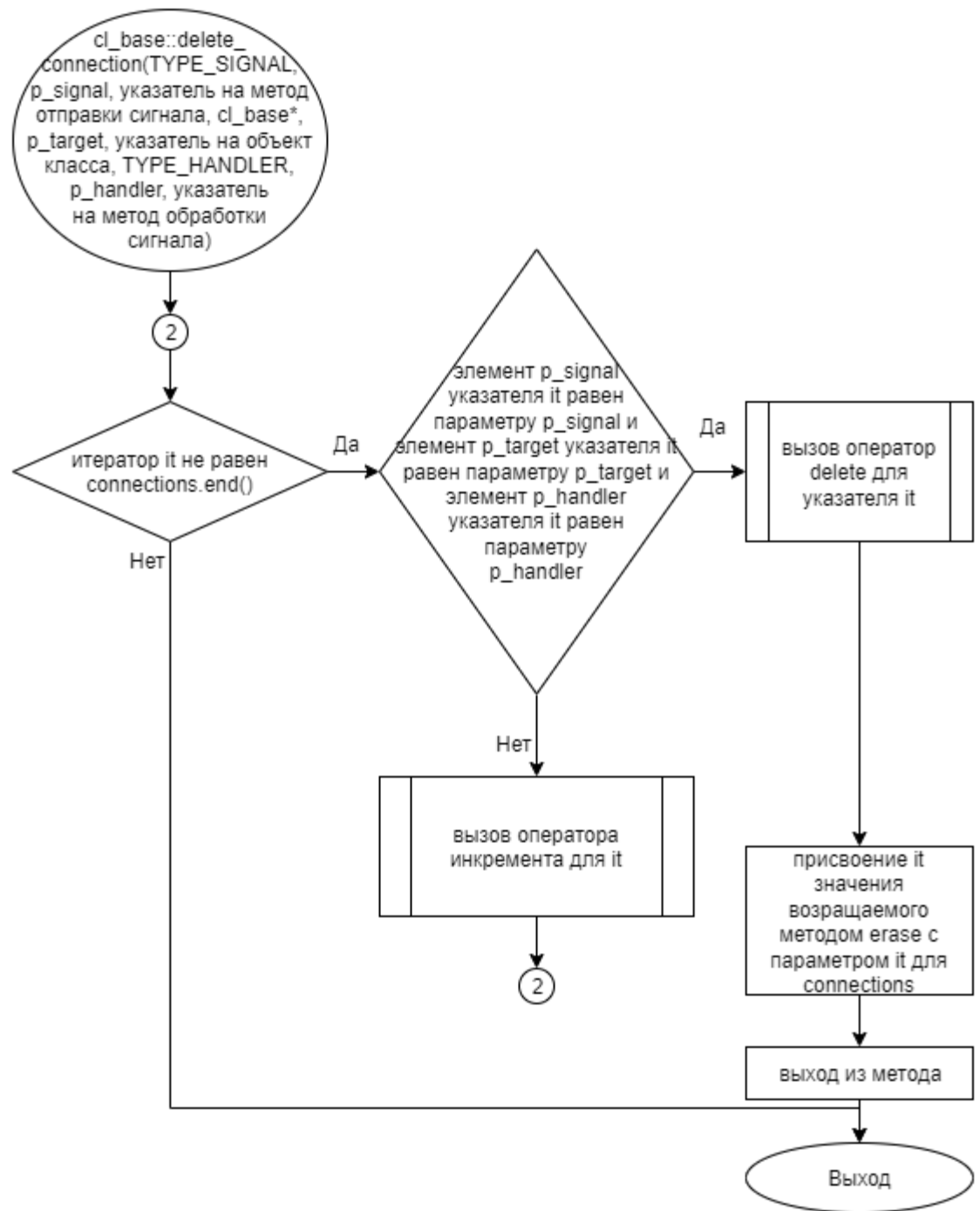


Рисунок 3 – Блок-схема алгоритма

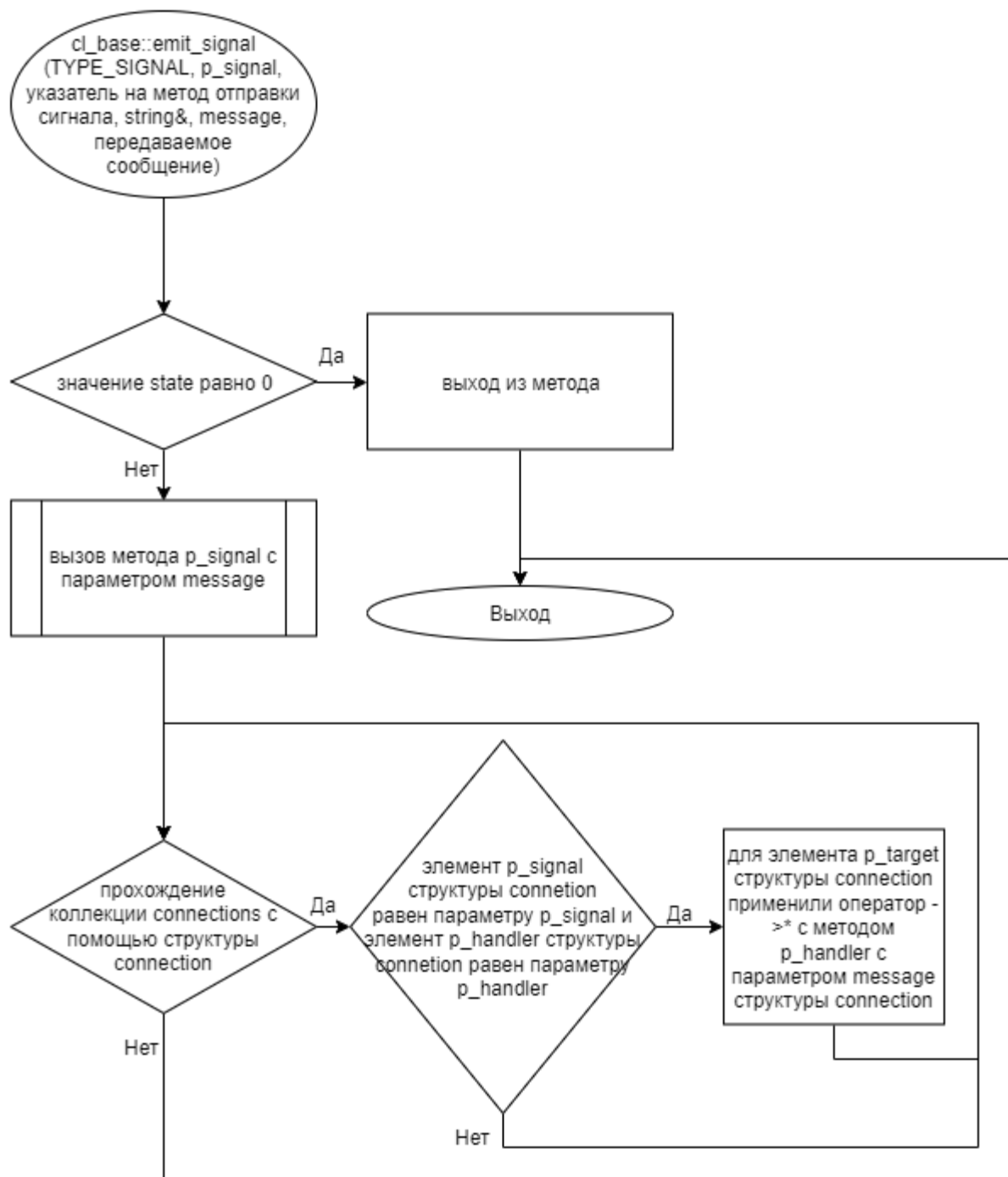


Рисунок 4 – Блок-схема алгоритма

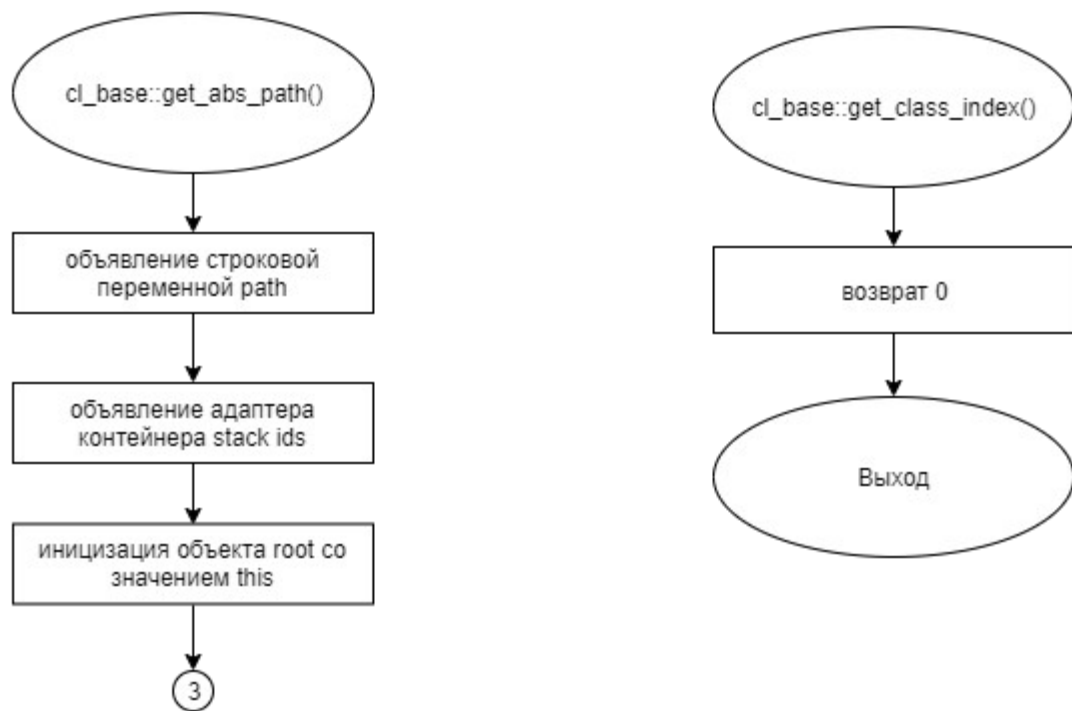


Рисунок 5 – Блок-схема алгоритма

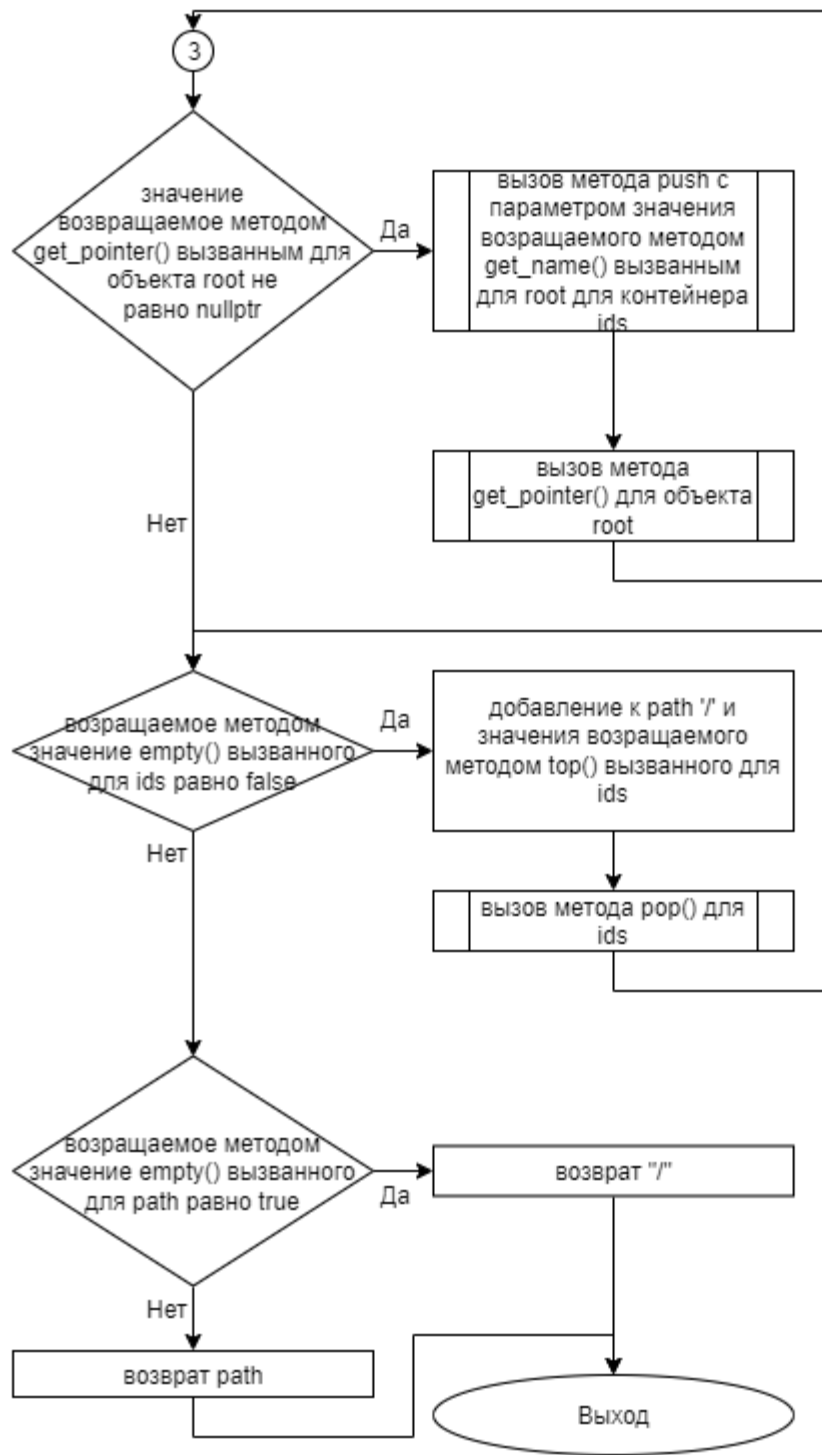


Рисунок 6 – Блок-схема алгоритма

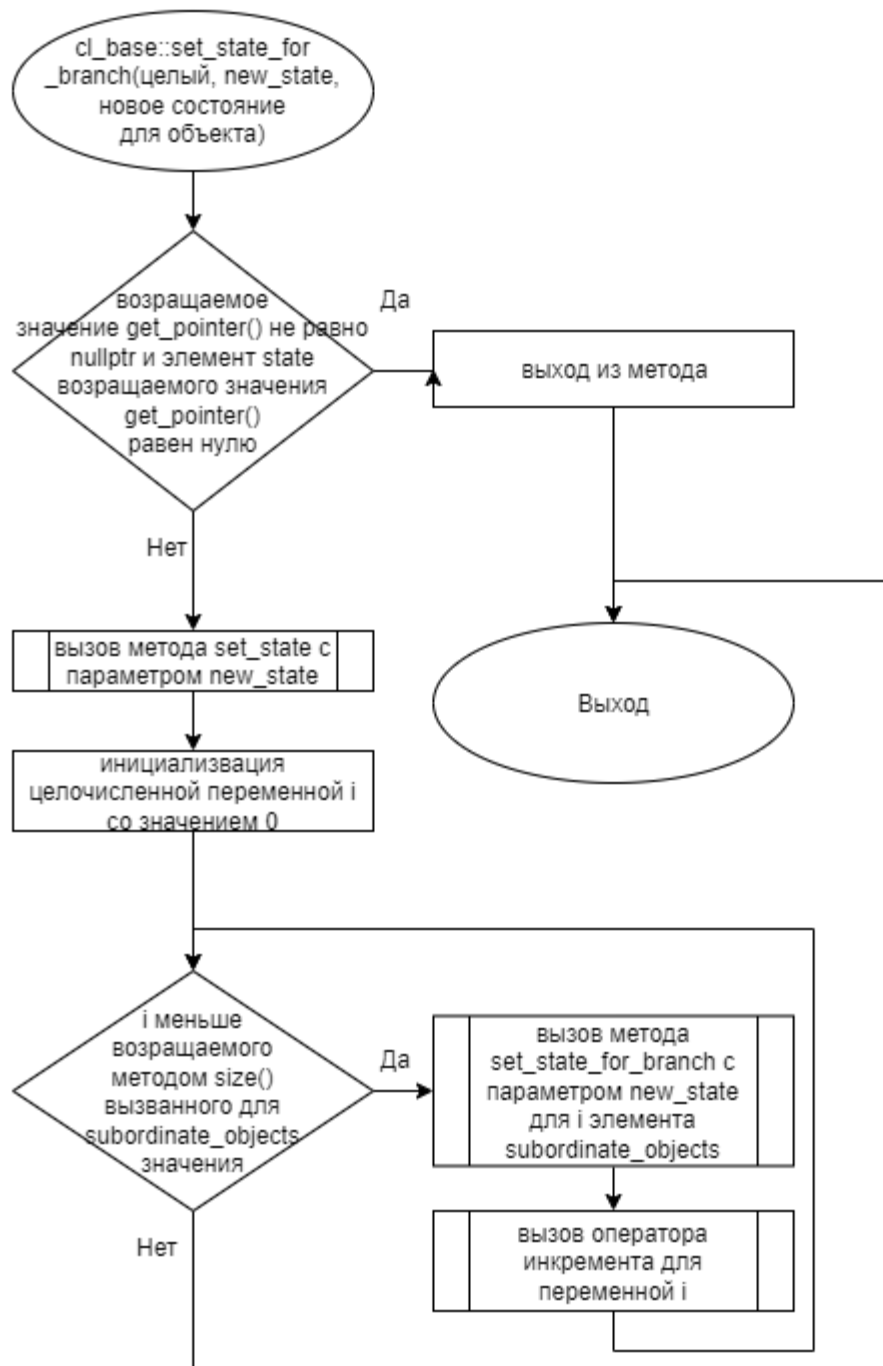


Рисунок 7 – Блок-схема алгоритма

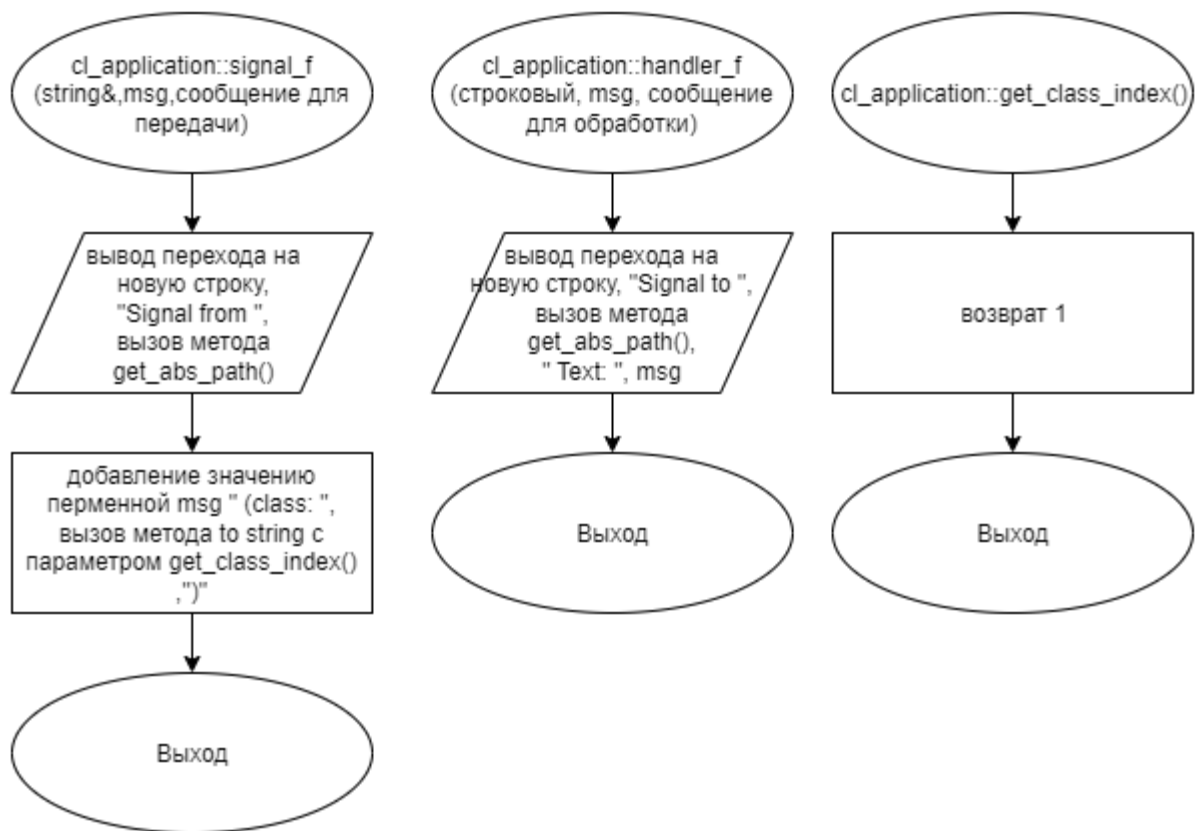


Рисунок 8 – Блок-схема алгоритма

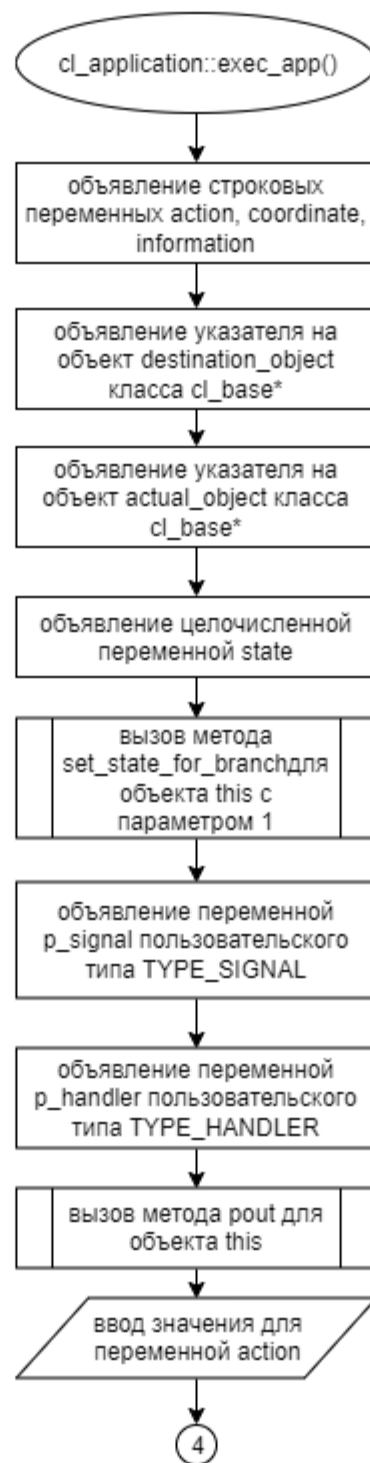


Рисунок 9 – Блок-схема алгоритма

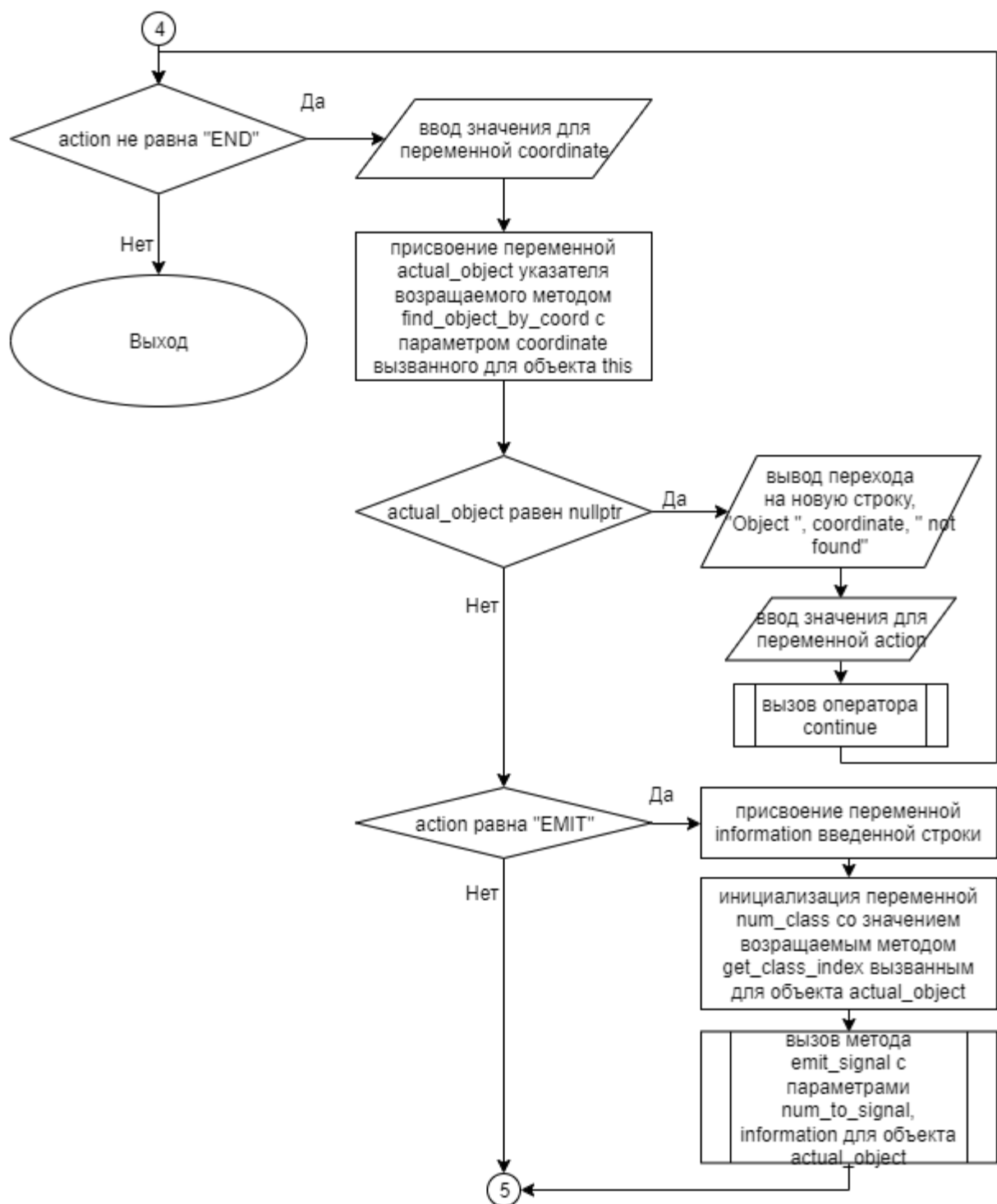


Рисунок 10 – Блок-схема алгоритма

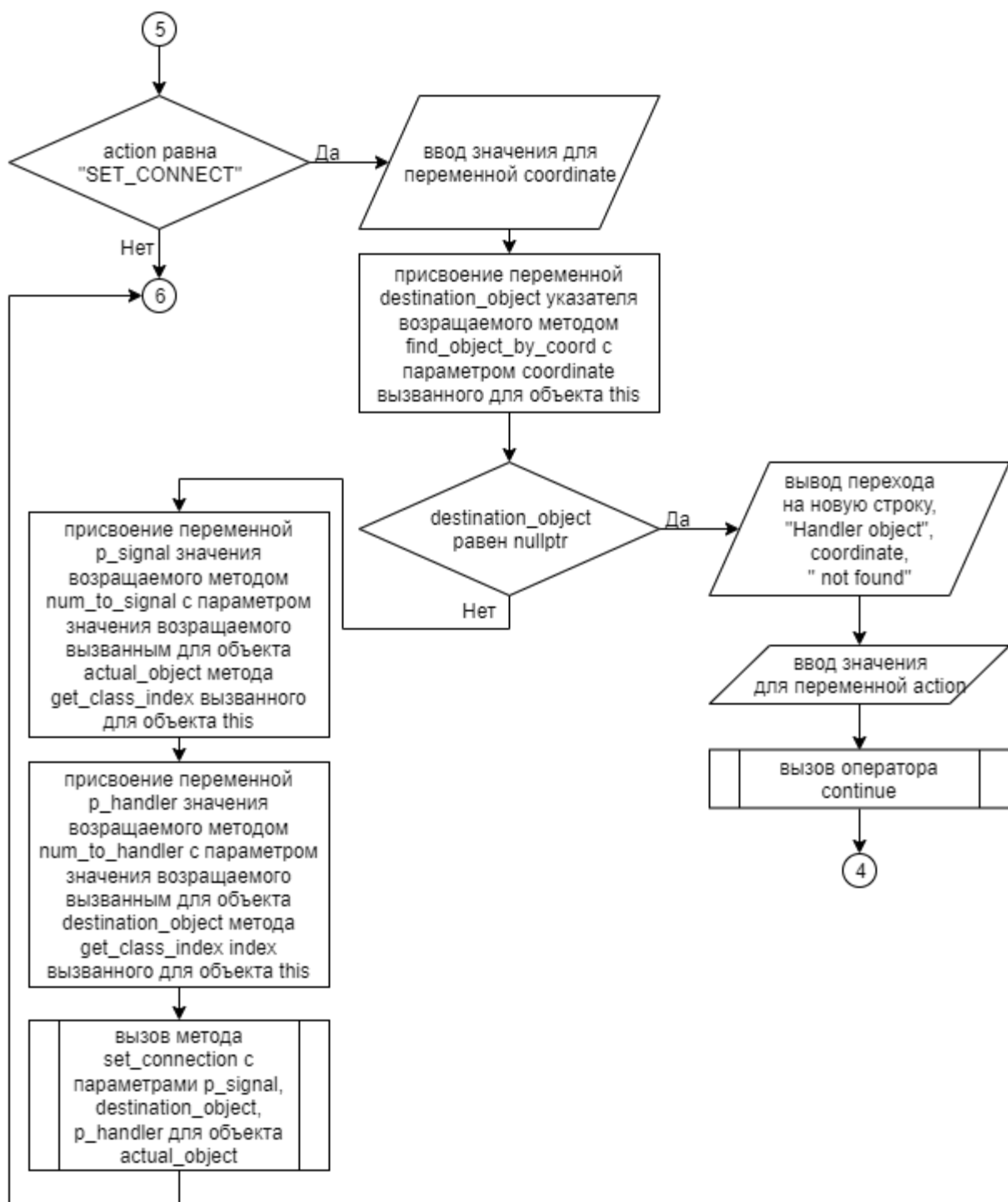


Рисунок 11 – Блок-схема алгоритма

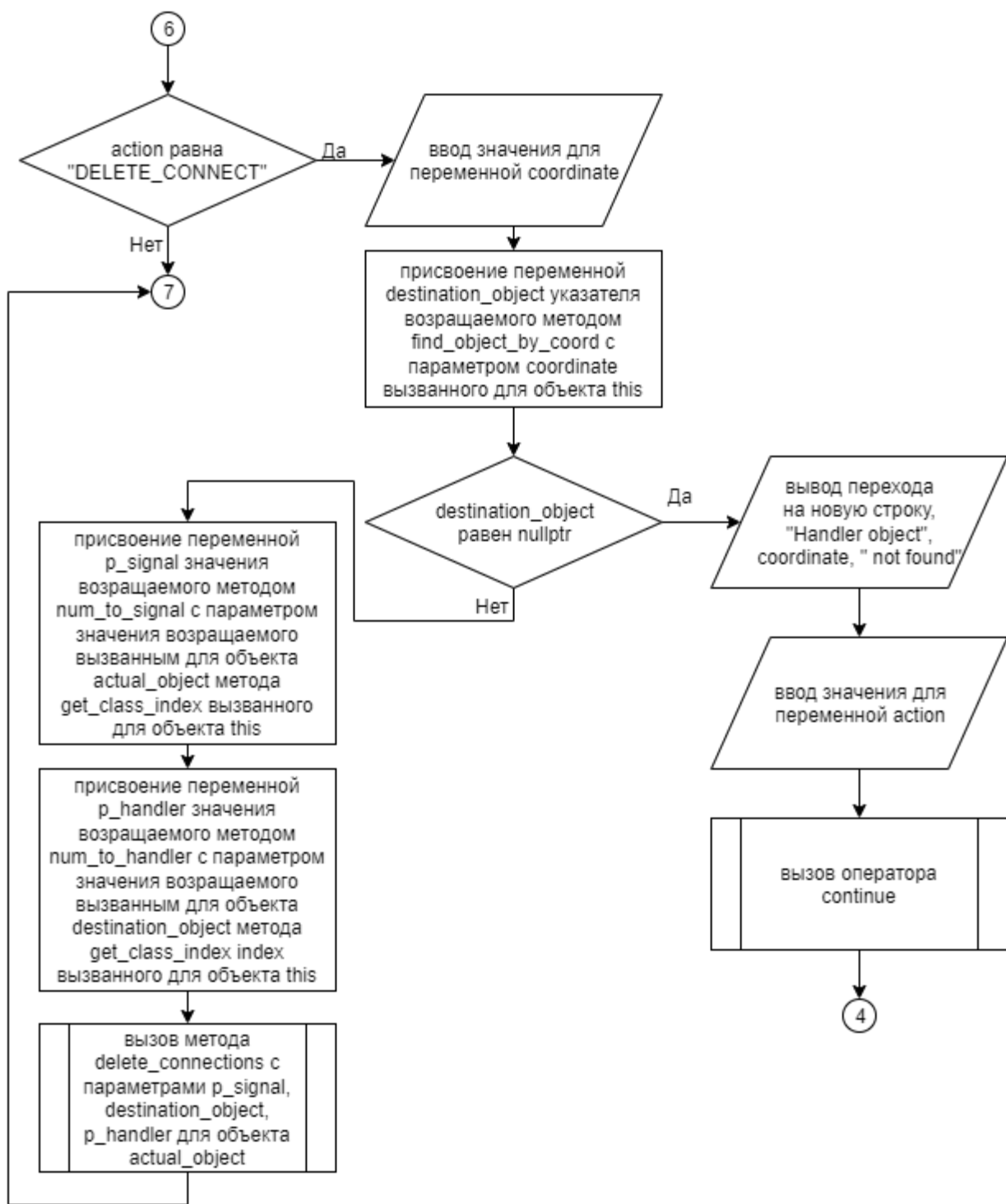


Рисунок 12 – Блок-схема алгоритма

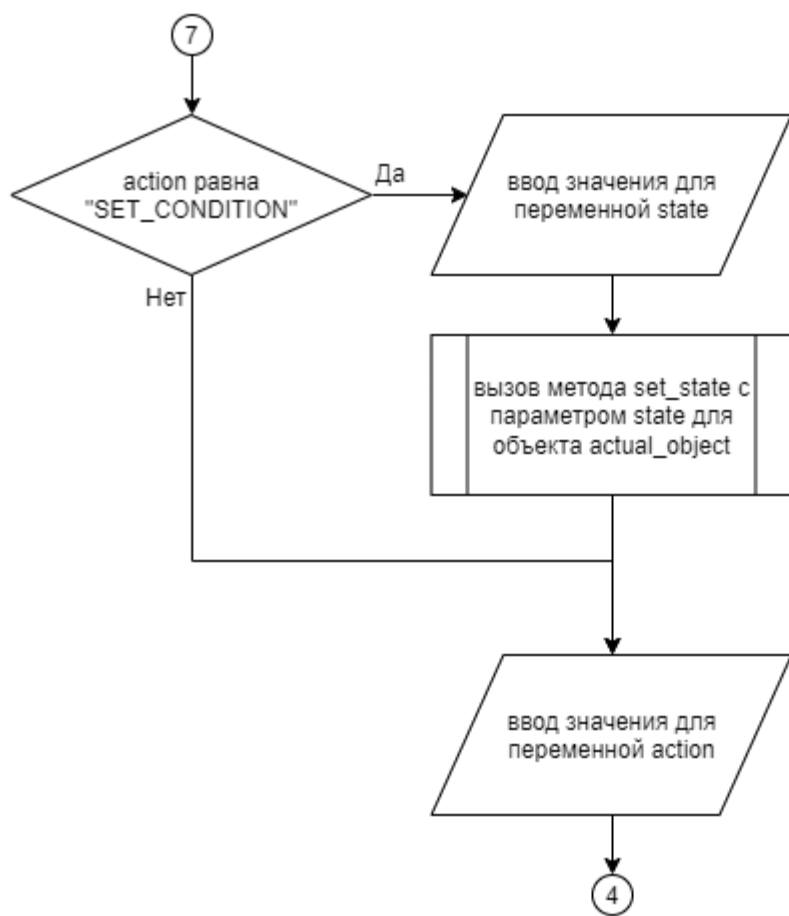


Рисунок 13 – Блок-схема алгоритма

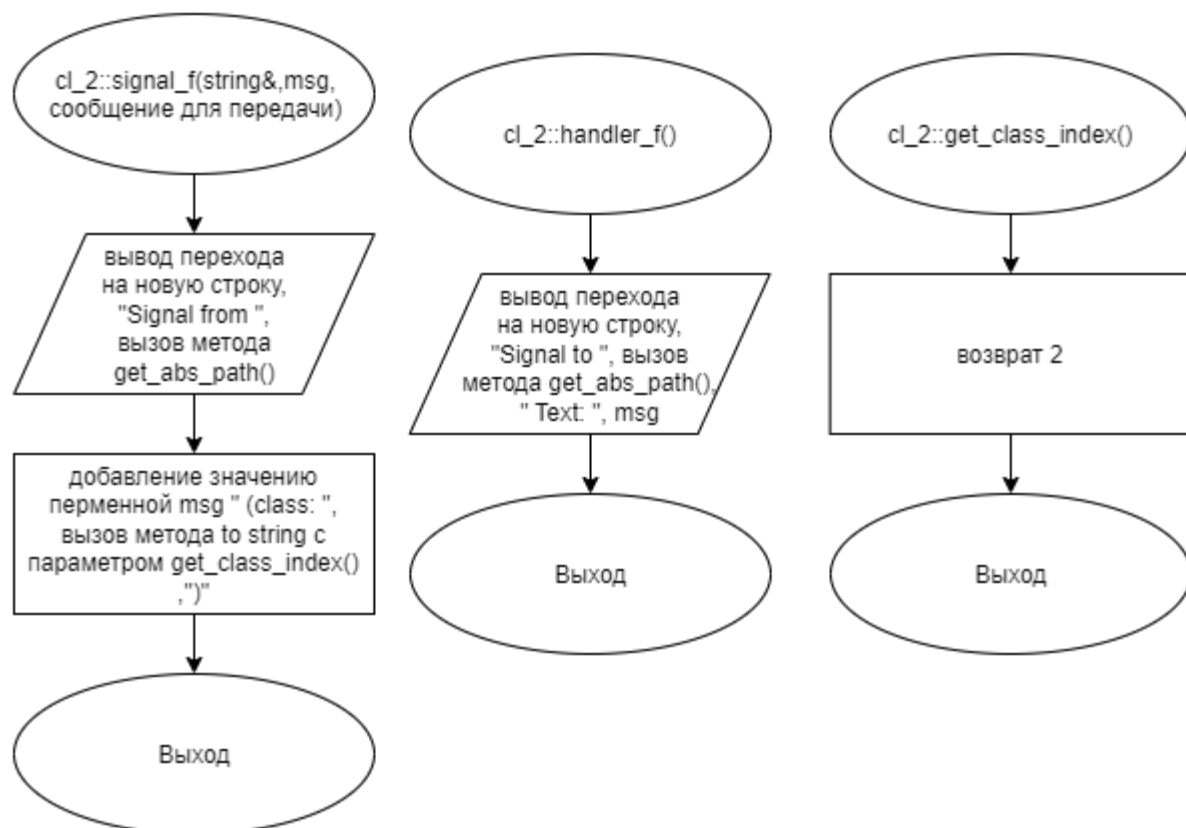


Рисунок 14 – Блок-схема алгоритма

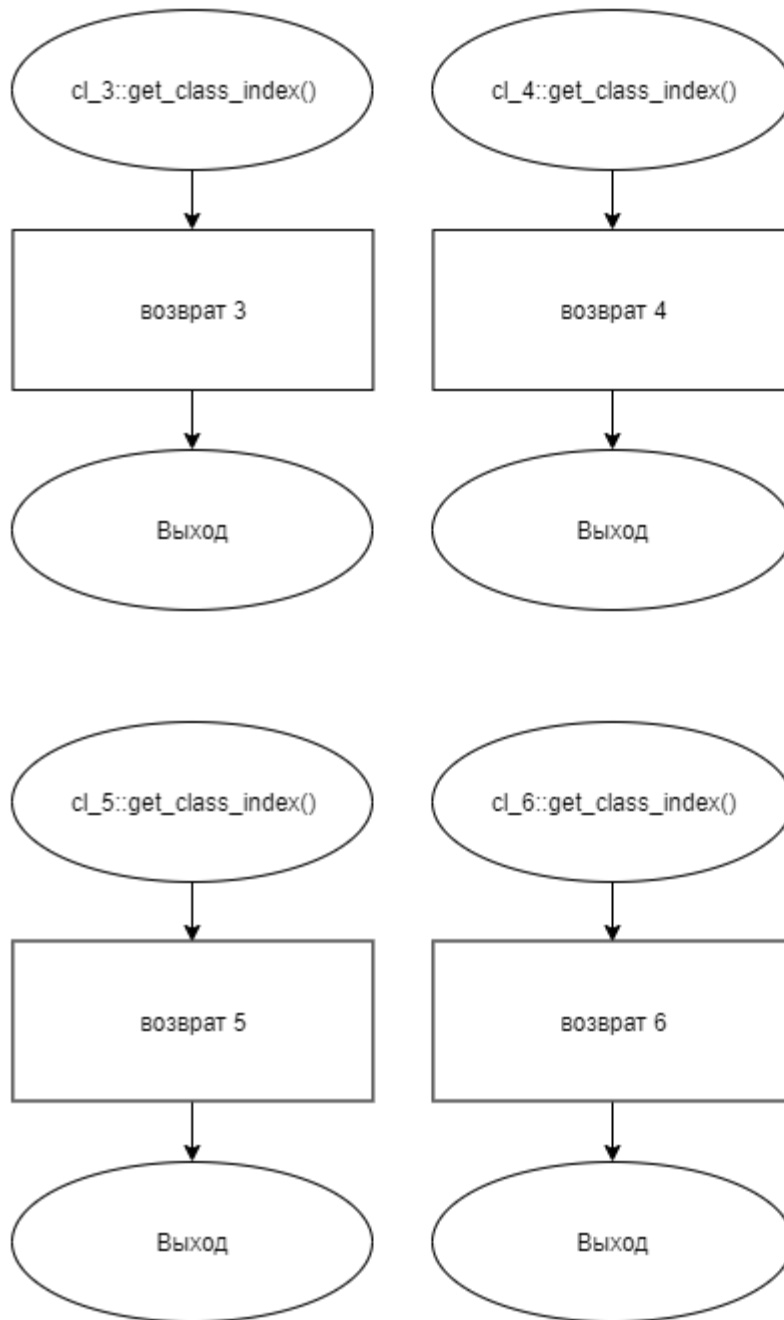


Рисунок 15 – Блок-схема алгоритма

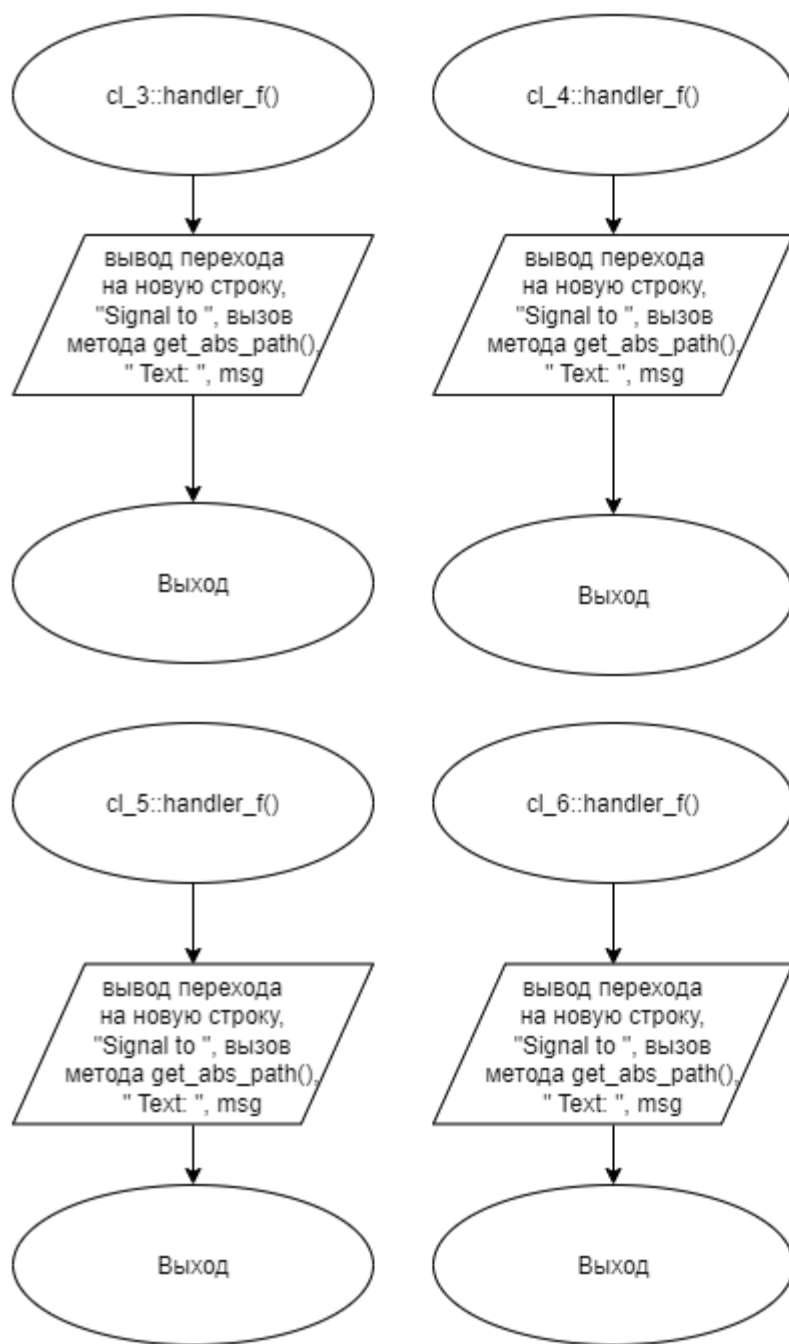


Рисунок 16 – Блок-схема алгоритма

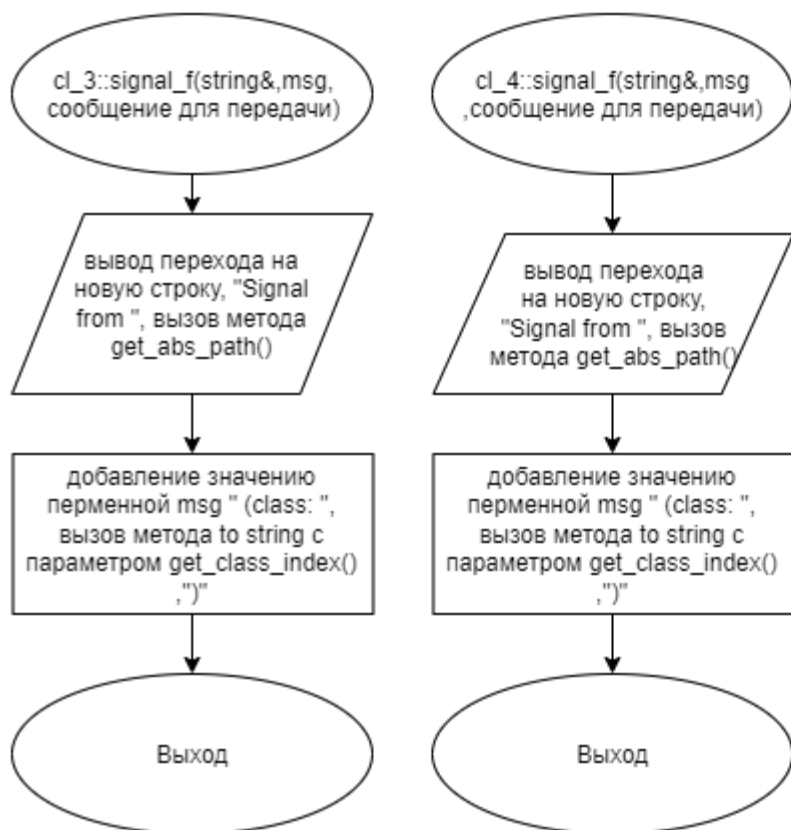


Рисунок 17 – Блок-схема алгоритма

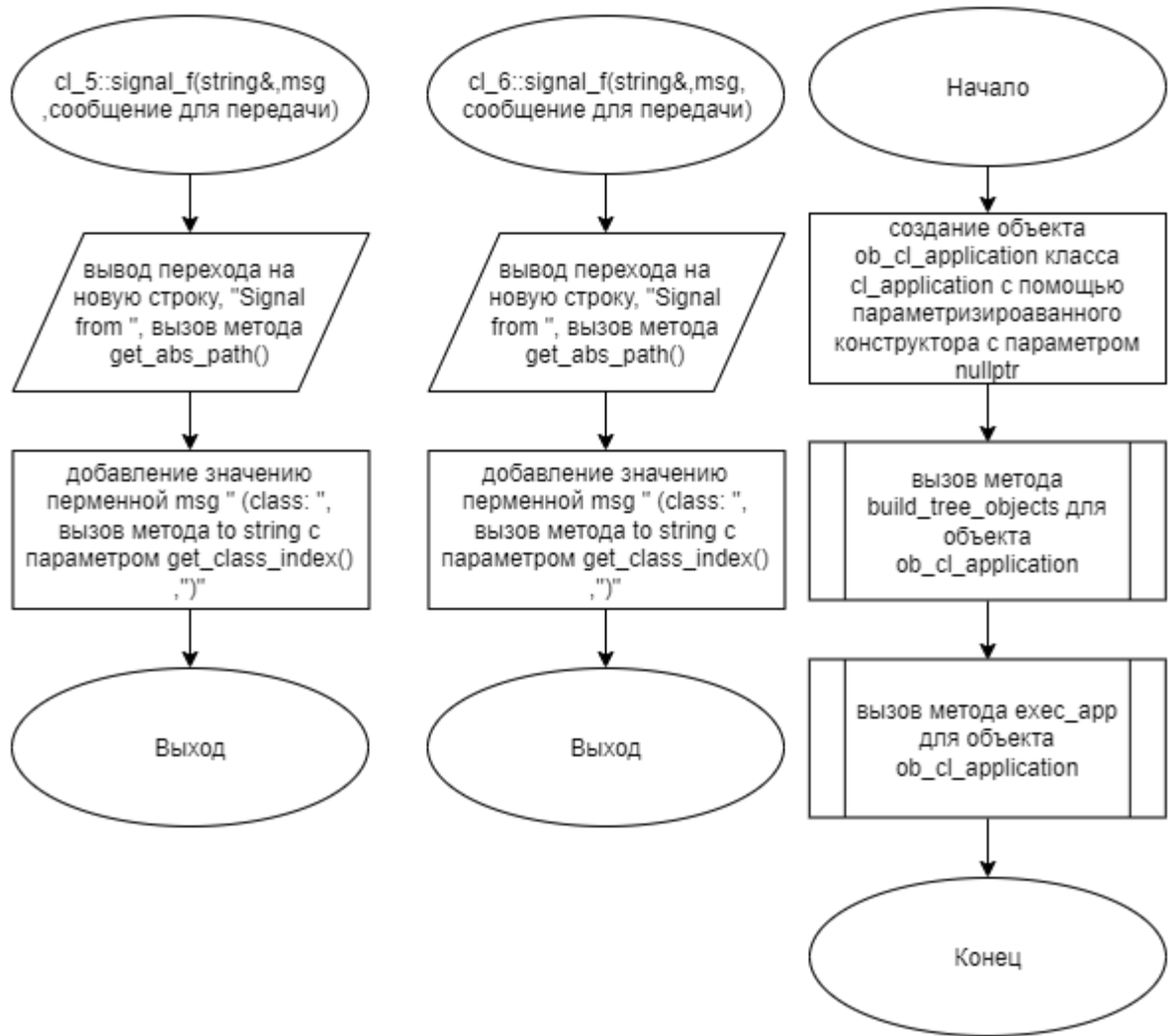


Рисунок 18 – Блок-схема алгоритма

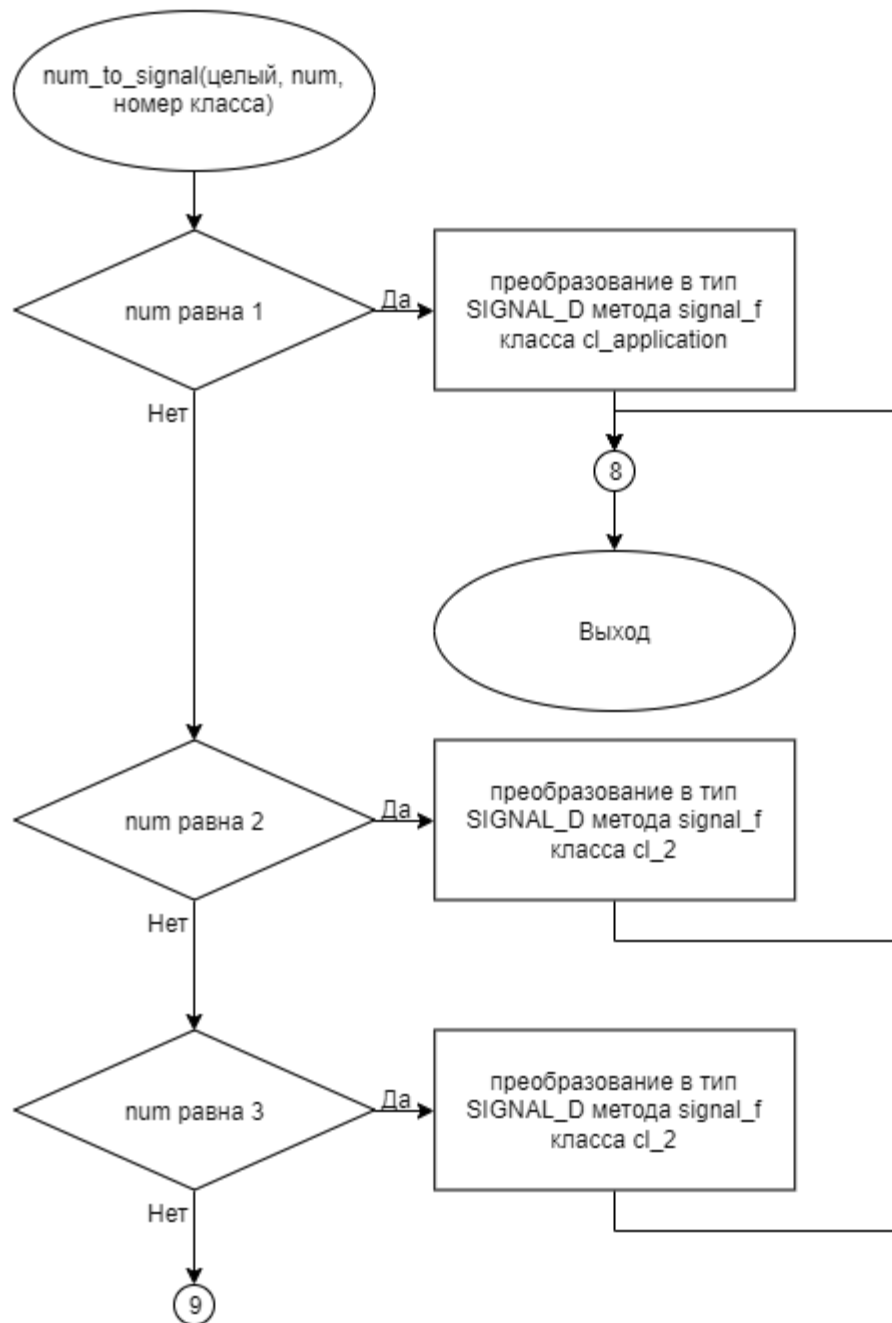


Рисунок 19 – Блок-схема алгоритма

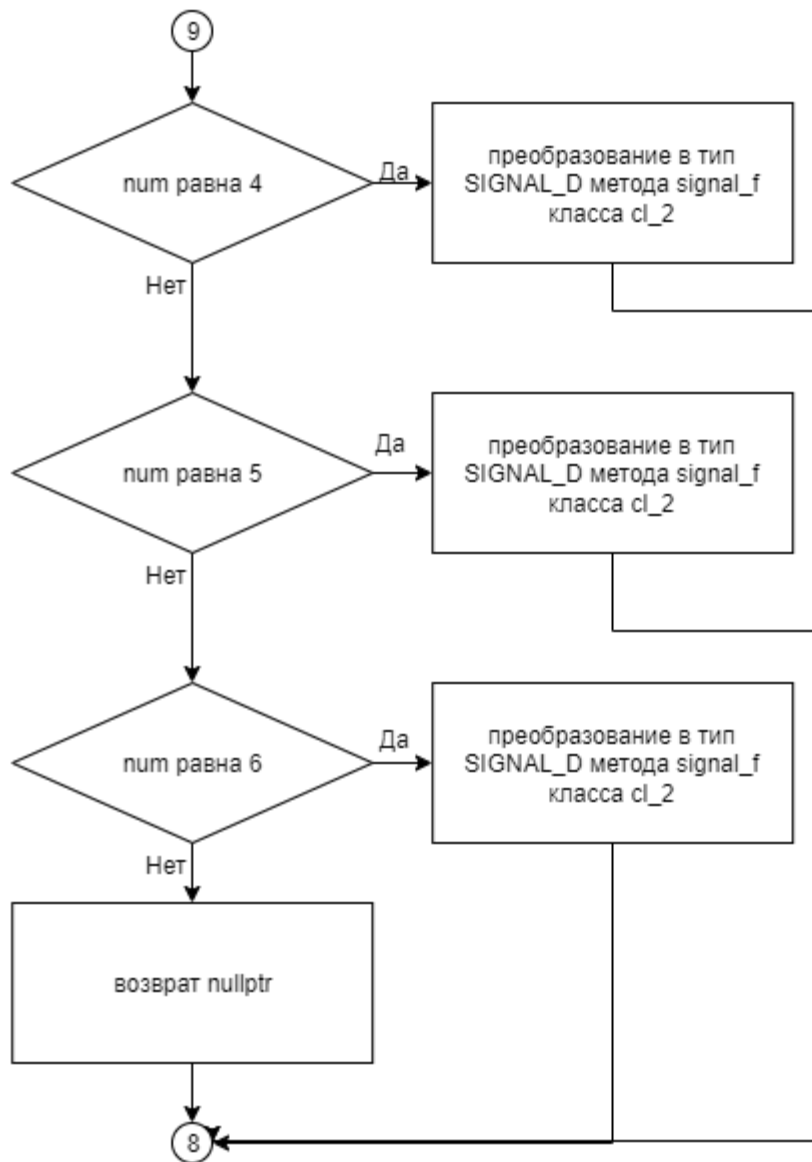


Рисунок 20 – Блок-схема алгоритма

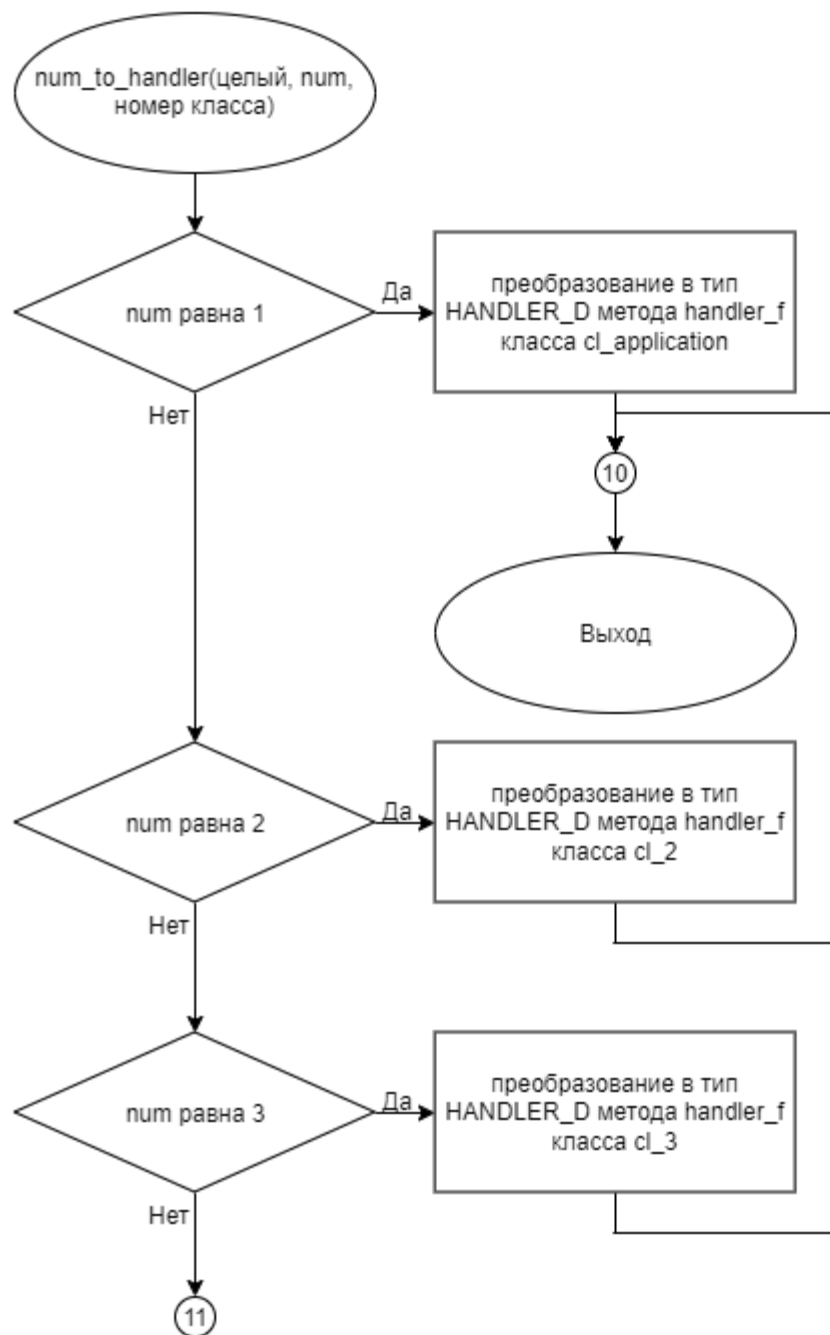


Рисунок 21 – Блок-схема алгоритма

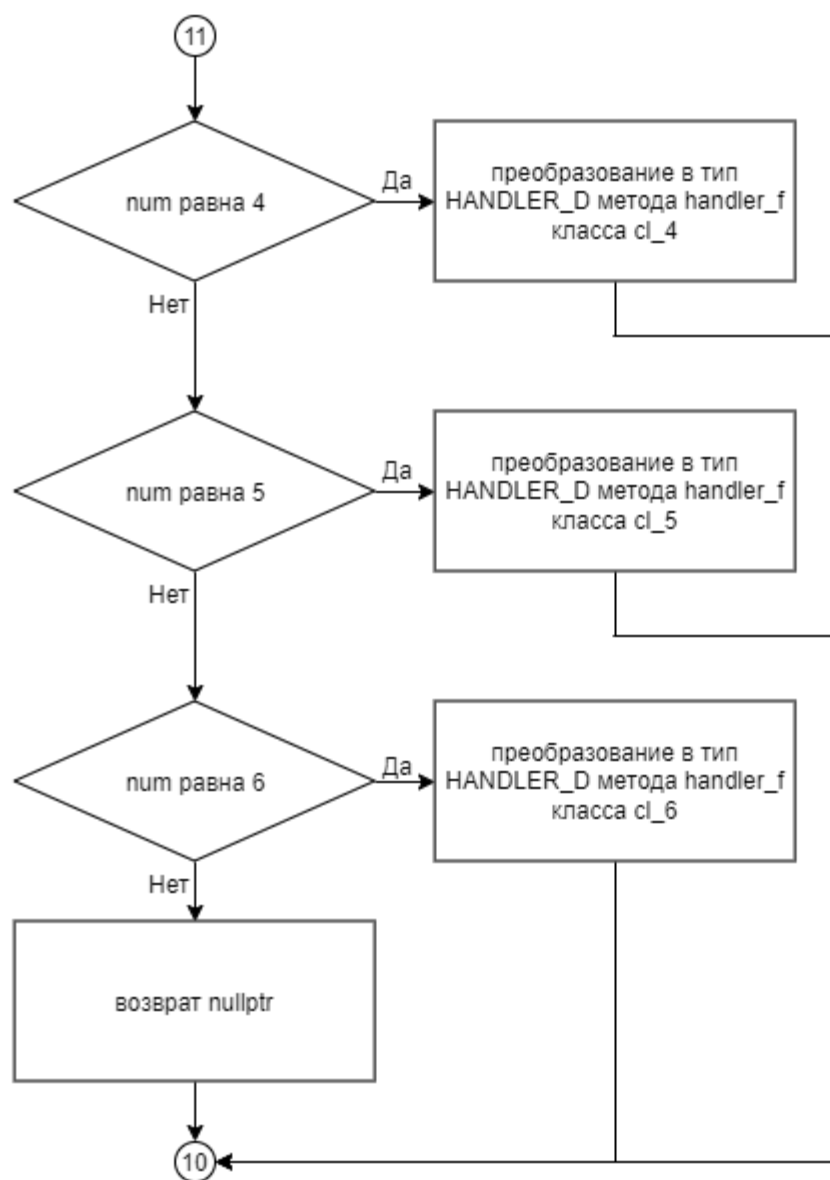


Рисунок 22 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_2.cpp

Листинг 1 – cl_2.cpp

```
#include "cl_2.h"

cl_2::cl_2(cl_base* p_head_object, const string
s_object_name):cl_base(p_head_object, s_object_name){
}
void cl_2::signal_f(string& msg){
    cout << endl << "Signal from " << get_abs_path();
    msg += " (class: "+ to_string(get_class_index()) + ")";
}
void cl_2::handler_f(string msg){
    cout << endl << "Signal to " << get_abs_path() << " Text: " << msg;
}
int cl_2::get_class_index(){
    return 2;
}
/*
Использование связей

cl_base* p_obj;
cl_base* p_target;
int i_cur_class = p_obj -> get_class_num();
int i_target_class = p_obj -> get_class_num();
switch(i_our_class)
{
    case 2:
        switch(i_target_class){
            case 2:
                p_obj -> set_connection (SIGNAL_D(cl_2:: signal_f), p_target,
HANDLER_D(cl_2::handler_f));
                break;

            case 3:
                p_obj -> set_connection (SIGNAL_D(cl_2:: signal_f), p_target,
HANDLER_D(cl_3::handler_f));
                break;
        }break;
    case 3:
        switch(i_target_class){
            case 2:
                p_obj -> set_connection (SIGNAL_D(cl_3:: signal_f), p_target,
```

```

HANDLER_D(cl_2::handler_f);
    break;

    case 3:
        p_obj -> set_connection (SIGNAL_D(cl_3:: signal_f), p_target,
HANDLER_D(cl_3::handler_f);
        break;
    }break;
}

или SIGNAL_D(cl_2:: signal_f) или (TYPE_SIGNAL)&cl_2:: signal_f
p_obj->set_connection(SIGNAL_D(cl_2:: signal_f), p_target,
HANDLER_D(cl_2::handler_f));
*/

```

5.2 Файл cl_2.h

Листинг 2 – cl_2.h

```

#ifndef __CL_2__H
#define __CL_2__H
#include "cl_base.h"
class cl_2 : public cl_base{
public:
    cl_2(cl_base* p_head_object, const string s_object_name = "Base_object");
    void signal_f(string& msg);
    void handler_f(string msg);
    int get_class_index();
};

#endif

```

5.3 Файл cl_3.cpp

Листинг 3 – cl_3.cpp

```

#include "cl_3.h"

cl_3::cl_3(cl_base* p_head_object, const string
s_object_name):cl_base(p_head_object, s_object_name){
}
void cl_3::signal_f(string& msg){
    cout << endl << "Signal from " << get_abs_path();
}

```

```

        msg += " (class: "+ to_string(get_class_index()) + ")";
    }
    void cl_3::handler_f(string msg){
        cout << endl << "Signal to " << get_abs_path() << " Text: " << msg;
    }
    int cl_3::get_class_index(){
        return 3;
    }

```

5.4 Файл cl_3.h

Листинг 4 – cl_3.h

```

#ifndef __CL_3__H
#define __CL_3__H
#include "cl_base.h"
class cl_3 : public cl_base{
public:
    cl_3(cl_base* p_head_object, const string s_object_name = "Base_object");
    int get_class_index();
    void signal_f(string& msg);
    void handler_f(string msg);
};

#endif

```

5.5 Файл cl_4.cpp

Листинг 5 – cl_4.cpp

```

#include "cl_4.h"

cl_4::cl_4(cl_base* p_head_object, const string
s_object_name):cl_base(p_head_object, s_object_name){

}
void cl_4::signal_f(string& msg){
    cout << endl << "Signal from " << get_abs_path();
    msg += " (class: "+ to_string(get_class_index()) + ")";
}
void cl_4::handler_f(string msg){
    cout << endl << "Signal to " << get_abs_path() << " Text: " << msg;
}
int cl_4::get_class_index(){

```

```
    return 4;
}
```

5.6 Файл cl_4.h

Листинг 6 – cl_4.h

```
#ifndef __CL_4__H
#define __CL_4__H
#include "cl_base.h"
class cl_4 : public cl_base{
public:
    cl_4(cl_base* p_head_object, const string s_object_name = "Base_object");
    int get_class_index();
    void signal_f(string& msg);
    void handler_f(string msg);
};

#endif
```

5.7 Файл cl_5.cpp

Листинг 7 – cl_5.cpp

```
#include "cl_5.h"

cl_5::cl_5(cl_base* p_head_object, const string
s_object_name):cl_base(p_head_object, s_object_name){
}
void cl_5::signal_f(string& msg){
    cout << endl << "Signal from " << get_abs_path();
    msg += " (class: "+ to_string(get_class_index()) + ")";
}
void cl_5::handler_f(string msg){
    cout << endl << "Signal to " << get_abs_path() << " Text: " << msg;
}
int cl_5::get_class_index(){
    return 5;
}
```

5.8 Файл cl_5.h

Листинг 8 – cl_5.h

```
#ifndef __CL_5__H
#define __CL_5__H
#include "cl_base.h"
class cl_5 : public cl_base{
public:
    cl_5(cl_base* p_head_object, const string s_object_name = "Base_object");
    int get_class_index();
    void signal_f(string& msg);
    void handler_f(string msg);
};

#endif
```

5.9 Файл cl_6.cpp

Листинг 9 – cl_6.cpp

```
#include "cl_6.h"

cl_6::cl_6(cl_base* p_head_object, const string
s_object_name):cl_base(p_head_object, s_object_name){
}
void cl_6::signal_f(string& msg){
    cout << endl << "Signal from " << get_abs_path();
    msg += " (class: "+ to_string(get_class_index()) + ")";
}
void cl_6::handler_f(string msg){
    cout << endl << "Signal to " << get_abs_path() << " Text: " << msg;
}
int cl_6::get_class_index(){
    return 6;
}
```

5.10 Файл cl_6.h

Листинг 10 – cl_6.h

```
#ifndef __CL_6__H
```

```

#define __CL_6__H
#include "cl_base.h"
class cl_6 : public cl_base{
public:
    cl_6(cl_base* p_head_object, const string s_object_name = "Base_object");
    int get_class_index();
    void signal_f(string& msg);
    void handler_f(string msg);
};

#endif

```

5.11 Файл cl_application.cpp

Листинг 11 – cl_application.cpp

```

#include "cl_application.h"
TYPE_SIGNAL num_to_signal(int num){
    switch(num){
        case 1:
            return SIGNAL_D(cl_application::signal_f);
        case 2:
            return SIGNAL_D(cl_2::signal_f);
        case 3:
            return SIGNAL_D(cl_3::signal_f);
        case 4:
            return SIGNAL_D(cl_4::signal_f);
        case 5:
            return SIGNAL_D(cl_5::signal_f);
        case 6:
            return SIGNAL_D(cl_6::signal_f);
    }
    return nullptr;
}
TYPE_HANDLER num_to_handler(int num){
    switch(num){
        case 1:
            return HANDLER_D(cl_application::handler_f);
        case 2:
            return HANDLER_D(cl_2::handler_f);
        case 3:
            return HANDLER_D(cl_3::handler_f);
        case 4:
            return HANDLER_D(cl_4::handler_f);
        case 5:
            return HANDLER_D(cl_5::handler_f);
        case 6:

```

```

        return HANDLER_D(cl_6::handler_f);
    }
    return nullptr;
}
void cl_application::build_tree_objects(){

    string path, name2;
    int i_class, i_state;
    cin >> path;
    setName(path);
    while(true){
        cin >> path;
        if(path == "endtree"){
            break;
        }
        cin >> name2 >> i_class;
        cl_base* parent = find_object_by_coord(path);
        //cout << "adamkdma1 " << parent << endl;
        if(parent == nullptr){
            cout << "Object tree";
            this -> pout();
            cout << endl << "The head object " << path << " is not found";
            exit(1);
        }
        if(parent ->get_child(name2) != nullptr){
            cout << endl << path << "Dubbing the names of subordinate objects";
        }
        else{
            if(i_class > 6) continue;
            switch(i_class){
                case 1:
                    new cl_application(parent, name2);
                    break;
                case 2:
                    new cl_2(parent,name2);
                    break;
                case 3:
                    new cl_3(parent,name2);
                    break;
                case 4:
                    new cl_4(parent,name2);
                    break;
                case 5:
                    new cl_5(parent,name2);
                    break;
                case 6:
                    new cl_6(parent,name2);
                    break;
            }
        }
    }

    /*
    string way, curname;

```

```

int i_class;
cout << "Object Tree";
cin >> curname;
setName(curname);
cl_base* p_head;
cl_base* p_sub;
cin >> way;
while(way != "endtree"){
    cin >> curname >> i_class;
    p_head = find_object_by_coord(way);
    if(!p_head){
        pout();
        cout << "\n" << "The head object " << way << "is not found";
        exit(1);
    }
    if(p_head->get_child(curname)){
        cout << "\n" << way << "Dubbing the names of subordinate objects";
    }
    else{
        switch(i_class){
            case 2:
                new cl_2(p_head, curname);
                break;
            case 3:
                new cl_3(p_head, curname);
                break;
            case 4:
                new cl_4(p_head, curname);
                break;
            case 5:
                new cl_5(p_head, curname);
                break;
            case 6:
                new cl_6(p_head, curname);
                break;
        }
    }
    cin >> way;
}
cl_base* destination_object;
cin >> way;
while(way != "end_of_connections"){
    cin >> curname;
    p_head = find_object_by_coord(way);
    destination_object = find_object_by_coord(curname);
    TYPE_SIGNAL signal_f = num_to_signal(p_head -> get_class_index());
    TYPE_HANDLER handler_f = num_to_handler(destination_object ->
get_class_index());
    p_head -> set_connection(signal_f, destination_object, handler_f);
    cin >> way;
}
*/
/*
vector <tuple<string, string, int>> objects_to_create;
while(true){

```

```

        cin >> path;
        if(path == "endtree") break;
        cin >> name2 >> i_class;
        objects_to_create.emplace_back(path, name2, i_class);
    }
    cout << "Object tree";
    //pout();cout << "Object tree";
    //pout();
    for(const auto& obj : objects_to_create){
        path = get<0>(obj);
        name2 = get<1>(obj);
        i_class = get<2>(obj);
        cl_base* parent = find_object_by_coord(path);
        //cout << "adamkdma1 " << parent << endl;
        if(parent == nullptr){
            this -> pout();
            cout << endl << "The head object " << path << " is not found";
            exit(1);
        }
        if(parent ->get_child(name2) != nullptr){
            cout << endl << path << "Dubbing the names of subordinate objects";
        }
        else{
            switch(i_class){
                case 1:
                    new cl_application(parent, name2);
                    break;
                case 2:
                    new cl_2(parent,name2);
                    break;
                case 3:
                    new cl_3(parent,name2);
                    break;
                case 4:
                    new cl_4(parent,name2);
                    break;
                case 5:
                    new cl_5(parent,name2);
                    break;
                case 6:
                    new cl_6(parent,name2);
                    break;
            }
        }
    }
    /*
}
int cl_application::exec_app(){

    string path;
    cl_base* targett;
    string target_path;

```

```

    cin >> path;
    while(path != "end_of_connections"){
    cin >> target_path;
    cl_base* parent = find_object_by_coord(path);
    targett = find_object_by_coord(target_path);
    TYPE_SIGNAL signal = num_to_signal(parent->get_class_index());
    TYPE_HANDLER handler = num_to_handler(targett->get_class_index());
    parent->set_connection(signal, targett, handler);
    cin >> path;
}

set_state_for_branch(1);
TYPE_SIGNAL signal;
TYPE_HANDLER handler;
cout << "Object tree";
pout();
int new_state;
string command, coord, msg;
cl_base* cur;
cl_base* target;
cin >> command;
while(command!="END"){
    cin >> coord;
    //cout <<endl<< "coord: " << coord;
    //if(coord=="END"){
    // break;
    //}
    cur = this -> find_object_by_coord(coord);
    if(cur == nullptr){
        cout << endl << "Object " << coord << " not found";
        cin >> command;
        continue;
    }
    if(command == "EMIT"){
        getline(cin, msg);
        int ind = cur -> get_class_index();
        cur -> emit_signal(num_to_signal(ind), msg);
    }
    else if(command == "SET_CONNECT"){
        cin >> coord;
        target = this -> find_object_by_coord(coord);
        if(target == nullptr){
            cout << endl << "Handler object " << coord << " not found";
            cin >> command;
            continue;
        }
        signal = num_to_signal(cur->get_class_index());
        handler = num_to_handler(target->get_class_index());
        cur -> set_connection(signal, target, handler);
    }
    else if(command == "DELETE_CONNECT"){

        cin >> coord;
        target = this -> find_object_by_coord(coord);
        //cout << endl << target->get_name();

```

```

        if(target == nullptr){
            cout << endl <<"Handler object " << coord << " not found";
            cin >> command;
            continue;
        }
        //cout << endl <<target->get_class_index()<< " " << cur-
>get_class_index();
        signal = num_to_signal(cur->get_class_index());
        handler = num_to_handler(target->get_class_index());
        cur -> delete_connection(signal, target, handler);
    }
    else if(command == "SET_CONDITION"){
        cin >> new_state;
        cur -> set_state(new_state);
    }
    cin >> command;

}
return 0;

//cout << endl << "The tree of objects and their readiness";
//pout_with_state();
//this->get_all_names();
}
*/
int cl_application::exec_app(){
    cout << "Object tree";
    string path;
    cl_base* targett;
    string target_path;
    cin >> path;
    while(path != "end_of_connections"){
        cin >> target_path;
        cl_base* parent = find_object_by_coord(path);
        targett = find_object_by_coord(target_path);
        TYPE_SIGNAL signal = num_to_signal(parent->get_class_index());
        TYPE_HANDLER handler = num_to_handler(targett->get_class_index());
        parent->set_connection(signal, targett, handler);
        cin >> path;
    }

    string action, coordinate, information;
    cl_base* destination_object;
    cl_base* actual_object;
    int state;
    this-> set_state_for_branch(1);
    TYPE_SIGNAL p_signal;
    TYPE_HANDLER p_handler;
    this->pout();
    cin>>action;
    while(action != "END"){
        cin >> coordinate;
        actual_object = this -> find_object_by_coord(coordinate);
        if(!actual_object){
            cout << endl << "Object " << coordinate << " not found";
            cin >> action;
        }
    }
}

```

```

        continue;
    }
    if(action == "EMIT")
    {
        getline(cin, information);
        int num_class = actual_object->get_class_index();
        actual_object->emit_signal(num_to_signal(num_class), information);
    }
    if(action == "SET_CONNECT")
    {
        cin >> coordinate;
        destination_object = this->find_object_by_coord(coordinate);
        if(!destination_object){
            cout << endl << "Handler object " << coordinate << " not found";
            cin >> action;
            continue;
        }
        p_signal = num_to_signal(actual_object -> get_class_index());
        p_handler = num_to_handler(destination_object->get_class_index());
        actual_object -> set_connection(p_signal, destination_object,
p_handler);
    }
    else if(action == "DELETE_CONNECT"){
        cin >> coordinate;
        destination_object = this -> find_object_by_coord(coordinate);
        if(!destination_object)
        {
            cout << endl << "Handler object " << coordinate << " not found";
            cin >> action;
            continue;
        }
        p_signal = num_to_signal(actual_object->get_class_index());
        p_handler = num_to_handler(destination_object->get_class_index());
        actual_object -> delete_connection(p_signal,
destination_object,p_handler);
    }
    else if(action == "SET_CONDITION"){
        cin >> state;
        actual_object->set_state(state);
    }
    cin >> action;
}
return 0;
}

cl_application::cl_application(cl_base* p_head_object,const string
s_object_name) : cl_base(p_head_object,s_object_name){};

void cl_application::signal_f(string& msg){
    cout << endl << "Signal from " << get_abs_path();
    msg += " (class: "+ to_string(get_class_index()) + ")";
}
void cl_application::handler_f(string msg){
    cout << endl << "Signal to " << get_abs_path() << " Text: " << msg;
}

```

```
int cl_application::get_class_index(){
    return 1;
}
```

5.12 Файл cl_application.h

Листинг 12 – cl_application.h

```
#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H
#include "cl_base.h"
#include "cl_2.h"
#include "cl_3.h"
#include "cl_4.h"
#include "cl_5.h"
#include "cl_6.h"
class cl_application : public cl_base{
public:
    void build_tree_objects();
    int exec_app();
    cl_application(cl_base* p_head_object,const string s_object_name =
"Base_object");
    void signal_f(string& msg);
    void handler_f(string msg);
    int get_class_index();
};

#endif
```

5.13 Файл cl_base.cpp

Листинг 13 – cl_base.cpp

```
#include "cl_base.h"

cl_base:: cl_base( cl_base* p_head_object,const string s_object_name){
    this -> p_head_object = p_head_object;
    this -> s_object_name = s_object_name;
    if( p_head_object != nullptr)
    {
        p_head_object -> subordinate_objects.push_back ( this );
    }
}

string cl_base:: get_name(){
```

```

    return this -> s_object_name;
}
cl_base* cl_base:: get_pointer(){
    return this->p_head_object;
}

cl_base* cl_base:: get_child(string name){
    for(auto it : this->subordinate_objects){
        if(it->get_name() == name){
            return it;
        }
    }
    return nullptr;
}

bool cl_base::setName(string new_name){
    if(get_pointer() != nullptr && get_child(new_name) != nullptr){
        return false;
    }
    s_object_name = new_name;
    return true;
}

cl_base::~~cl_base(){
    for(auto it : subordinate_objects){
        delete it;
    }
    subordinate_objects.clear();
}

cl_base* cl_base::branch_search(string name){

    cl_base* found = nullptr;
    queue<cl_base*> objects;
    objects.push(this);
    while(!objects.empty()){
        cl_base* front = objects.front();
        objects.pop();
        if(front-> get_name() == name){
            if(found == nullptr){
                found = front;
            }
            else{
                return nullptr;
            }
        }
        for(auto it : front->subordinate_objects){
            objects.push(it);
        }
    }
    return found;
}

cl_base* cl_base::tree_search(string name){

```

```

        cl_base* root = this;
        while(root->get_pointer() != nullptr){
            root = root ->get_pointer();
        }
        return root->branch_search(name);
    }
    void cl_base::set_state(int new_state){
        if(new_state != 0){
            state = new_state;
            cl_base* root = this;
            while(root->get_pointer() != nullptr){
                root = root ->get_pointer();
                if(root->state == 0){
                    this->state = 0;
                    break;
                }
            }
        }
        else{
            state = 0;
            for(auto it : subordinate_objects){
                it->set_state(new_state);
            }
        }
    }
    void cl_base::pout(int indent){
        cout << endl << string(indent, ' ') << get_name();
        for(auto it : subordinate_objects){
            it->pout(indent + 4);
        }
    }
    void cl_base::pout_with_state(int indent){
        cout << endl << string(indent, ' ') << get_name();
        if(state != 0){
            cout << " is ready";
        }
        if(state == 0){
            cout << " is not ready";
        }
        for(auto it : subordinate_objects){
            it->pout_with_state(indent + 4);
        }
    }
    cl_base* cl_base::find_object_by_coord(string coord){
        if(coord.empty()){
            return nullptr;
        }
        if(coord == "/"){
            cl_base* root = this;
            while(root -> p_head_object != nullptr){
                root = root->p_head_object;
            }
            return root;
        }
    }

```

```

    }
    if(coord.substr(0, 2) == "//"){
        string name = coord.substr(2);
        return tree_search(name);
    }
    if(coord== "."){
        return this;
    }
    if(coord[0] == '.'){
        string name = coord.substr(1);
        return branch_search(name);
    }
    if(coord[0] != '/'){
        size_t pos = coord.find('/');
        string first = (pos == string::npos) ? coord : coord.substr(0, pos);
        cl_base* child = get_child(first);
        if(child == nullptr) return nullptr;
        if(pos == string::npos) return child;
        return child->find_object_by_coord(coord.substr(pos + 1));
    }
    cl_base* root = this;
    while(root->p_head_object != nullptr){
        root = root->p_head_object;
    }
    if(coord == "/") return root;
    return root -> find_object_by_coord(coord.substr(1));
}

void cl_base::delete_sub_object(string name){

    vector <cl_base*> & new_children = this->subordinate_objects;
    for(int i = 0; i< new_children.size();i++){
        if(new_children[i]->get_name()== name){
            delete new_children[i];
            new_children.erase(new_children.begin() + i);
            return;
        }
    }

}

bool cl_base::change_head(cl_base* new_head)
{

    cl_base* root = this;
    while(root->get_pointer() != nullptr){
        root = root->get_pointer();
    }
    if(this == root || new_head == nullptr){
        //cout << "    rr    " << new_head << endl;
        return false;
    }
    if(new_head -> get_child(this->s_object_name) != nullptr){

        return false;
    }
}

```

```

        cl_base* obj = new_head;
        while(obj != nullptr){
            if(obj == this)return false;
            obj = obj -> get_pointer();
        }
        for(int i = 0; i < p_head_object -> subordinate_objects.size(); i++){
            if(p_head_object -> subordinate_objects[i] == this){
                p_head_object -> subordinate_objects.erase(p_head_object ->
subordinate_objects.begin() + i);
                break;
            }
        }
        this -> p_head_object = new_head;
        new_head -> subordinate_objects.push_back(this);
        return true;
    }

void cl_base:: set_connection(TYPE_SIGNAL    p_signal,  cl_base*  p_target,
TYPE_HANDLER p_handler){
    o_sh* p_value;
    for(int i = 0; i < connections.size(); i++){
        if(connections[i] -> p_signal == p_signal && connections[i] -> p_target
== p_target && connections[i] -> p_handler == p_handler){
            return;
        }
    }
    p_value = new o_sh();
    p_value -> p_signal = p_signal;
    p_value -> p_target = p_target;
    p_value -> p_handler = p_handler;
    connections.push_back(p_value);
};

void cl_base:: delete_connection(TYPE_SIGNAL    p_signal,  cl_base*  p_target,
TYPE_HANDLER p_handler){

    for(auto it = connections.begin();it != connections.end();){
        if((*it) -> p_signal == p_signal && (*it) -> p_target == p_target &&
(*it) -> p_handler == p_handler){
            delete *it;
            it = connections.erase(it);
            return;
        }
        else{
            ++it;
        }
    }
    //connections.erase(remove_if(connections.begin(),  connections.end(),[&]
(const o_sh* conn){return conn -> p_signal == p_signal && conn -> p_target
== p_target && conn->p_handler == p_handler;}),connections.end());
}

void cl_base:: emit_signal(TYPE_SIGNAL    p_signal,  string& message){
    if(this -> state == 0){
        return;
    }
}

```

```

        (this->*p_signal)(message);
        for(auto& connection : connections){
            if(connection->p_signal == p_signal && connection->p_target->state !=
0){
                (connection->p_target->*connection->p_handler)(message);
            }
        }
    };
    string cl_base:: get_abs_path(){
        string path;
        stack<string> ids;
        cl_base* root = this;
        while(root->get_pointer() != nullptr){
            ids.push(root->get_name());
            root = root->get_pointer();
        }
        while(!ids.empty()){
            path += '/' + ids.top();
            ids.pop();
        }
        if(path.empty()){
            return "/";
        }
        return path;
    }
    int cl_base::get_class_index(){
        return 0;
    }

    void cl_base:: set_state_for_branch(int new_state){
        if(get_pointer() != nullptr && get_pointer()->state == 0){
            return;
        }
        set_state(new_state);
        for(int i = 0; i < subordinate_objects.size();i++){
            subordinate_objects[i]->set_state_for_branch(new_state);
        }
    }
}

```

5.14 Файл cl_base.h

Листинг 14 – cl_base.h

```

#ifndef __CL_BASE__H
#define __CL_BASE__H
#include<iostream>
#include<vector>
#include<queue>
#include<stack>

```

```

#define SIGNAL_D(signal_f)static_cast<TYPE_SIGNAL>(&signal_f)
#define HANDLER_D(handler_f)static_cast<TYPE_HANDLER>(&handler_f)
using namespace std;
class cl_base;
typedef void ( cl_base :: *TYPE_SIGNAL ) ( string & );
typedef void ( cl_base :: *TYPE_HANDLER ) ( string );
struct o_sh
{
    TYPE_SIGNAL p_signal;
    cl_base* p_target;
    TYPE_HANDLER p_handler;
};

class cl_base{
    int state = 0;
    cl_base* p_head_object;
    vector < cl_base* > subordinate_objects;
    string s_object_name;
    vector <o_sh*> connections;
public:
    ~cl_base();
    cl_base( cl_base* p_head_object,const string s_object_name = "Base_object"
);
    string get_name();
    cl_base* get_pointer();
    //void get_all_names();
    cl_base* branch_search(string name);
    cl_base* tree_search(string name);
    //2
    void pout(int indent = 0);
    void pout_with_state(int indent = 0);
    void set_state(int new_state);
    bool setName(string new_name);
    cl_base* get_child(string name);
    //3
    cl_base* find_object_by_coord(string coord);
    bool change_head(cl_base* new_head);
    void delete_sub_object(string name);

    //4
    void set_connection(TYPE_SIGNAL p_signal, cl_base* p_target, TYPE_HANDLER
p_handler);
    void delete_connection(TYPE_SIGNAL p_signal, cl_base* p_target,
TYPE_HANDLER p_handler);
    void emit_signal(TYPE_SIGNAL p_signal, string& message);
    string get_abs_path();
    virtual int get_class_index();
    void set_state_for_branch(int new_state);
};

#endif

```

5.15 Файл main.cpp

Листинг 15 – main.cpp

```
#include "cl_application.h"
int main()
{
    cl_application  ob_cl_application( nullptr );
    ob_cl_application.build_tree_objects ( );
    return ob_cl_application.exec_app();
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 30.

Таблица 30 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
<pre> appls_root / object_s1 3 / object_s2 2 /object_s2 object_s4 4 / object_s13 5 /object_s2 object_s6 6 /object_s1 object_s7 2 endtree /object_s2/object_s4 /object_s2/object_s6 /object_s2 /object_s1/object_s7 / /object_s2/object_s4 /object_s2/object_s4 / end_of_connections EMIT /object_s2/object_s4 Send message 1 EMIT /object_s2/object_s4 Send message 2 EMIT /object_s2/object_s4 Send message 3 EMIT /object_s1 Send message 4 END </pre>	<pre> Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1 </pre>	<pre> Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1 </pre>
<pre> root / A 2 / B 3 endtree / /A /A /B </pre>	<pre> Object tree root A B Signal from / Signal from / </pre>	<pre> Object tree root A B Signal from / Signal from / </pre>

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
<pre> end_of_connections SET_CONDITION /A 0 EMIT / Test message SET_CONDITION /A 1 EMIT / Test again END </pre>	<pre> Signal to /A Text: Test again (class: 1) </pre>	<pre> Signal to /A Text: Test again (class: 1) </pre>
<pre> root / A 2 / B 3 endtree / /A end_of_connections SET_CONNECT / /Z END </pre>	<pre> Object tree root A B Handler object /Z not found </pre>	<pre> Object tree root A B Handler object /Z not found </pre>
<pre> appls_root / obj_X 4 / obj_Y 5 endtree /obj_X /obj_Y /obj_X /obj_Y end_of_connections EMIT /obj_X "Message before delete" DELETE_CONNECT /obj_X /obj_Y EMIT /obj_X "Message after delete" END </pre>	<pre> Object tree appls_root obj_X obj_Y Signal from /obj_X Signal to /obj_Y Text: "Message before delete" (class: 4) Signal from /obj_X </pre>	<pre> Object tree appls_root obj_X obj_Y Signal from /obj_X Signal to /obj_Y Text: "Message before delete" (class: 4) Signal from /obj_X </pre>

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).